

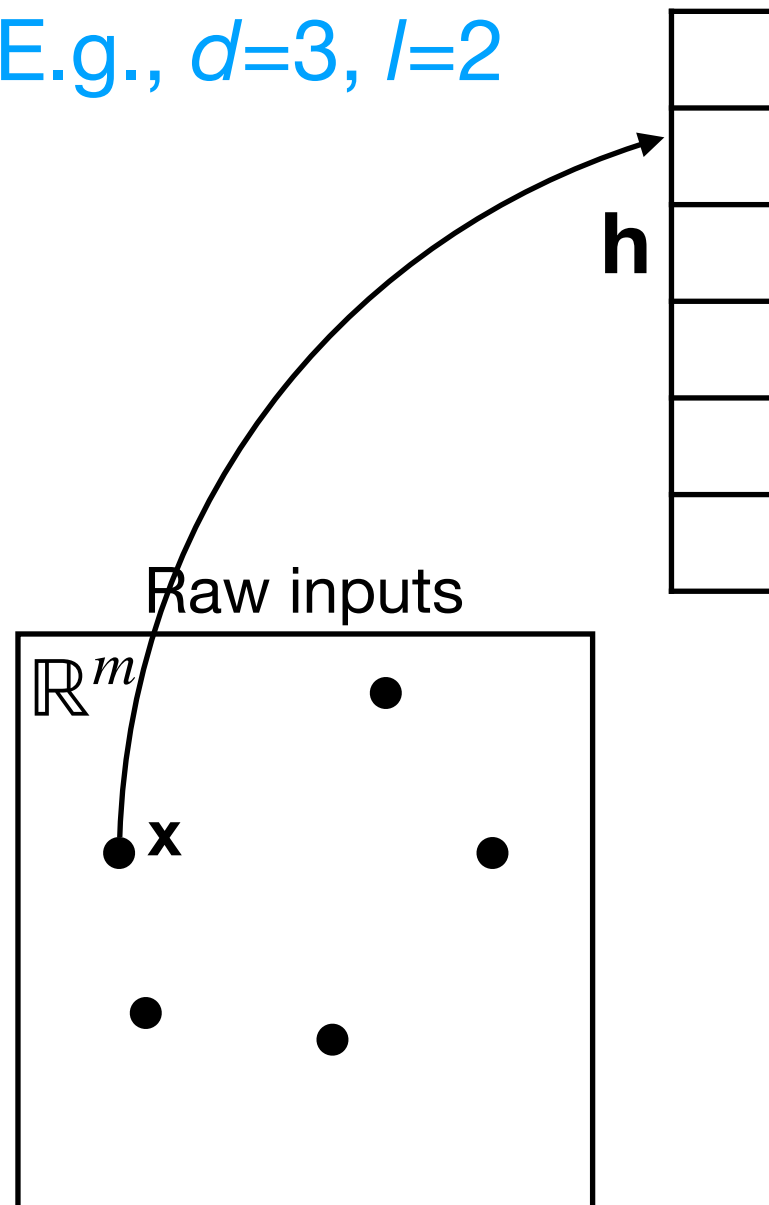
# CS/DS 552: Class 9

Jacob Whitehill

# (Deep) VQ-VAEs

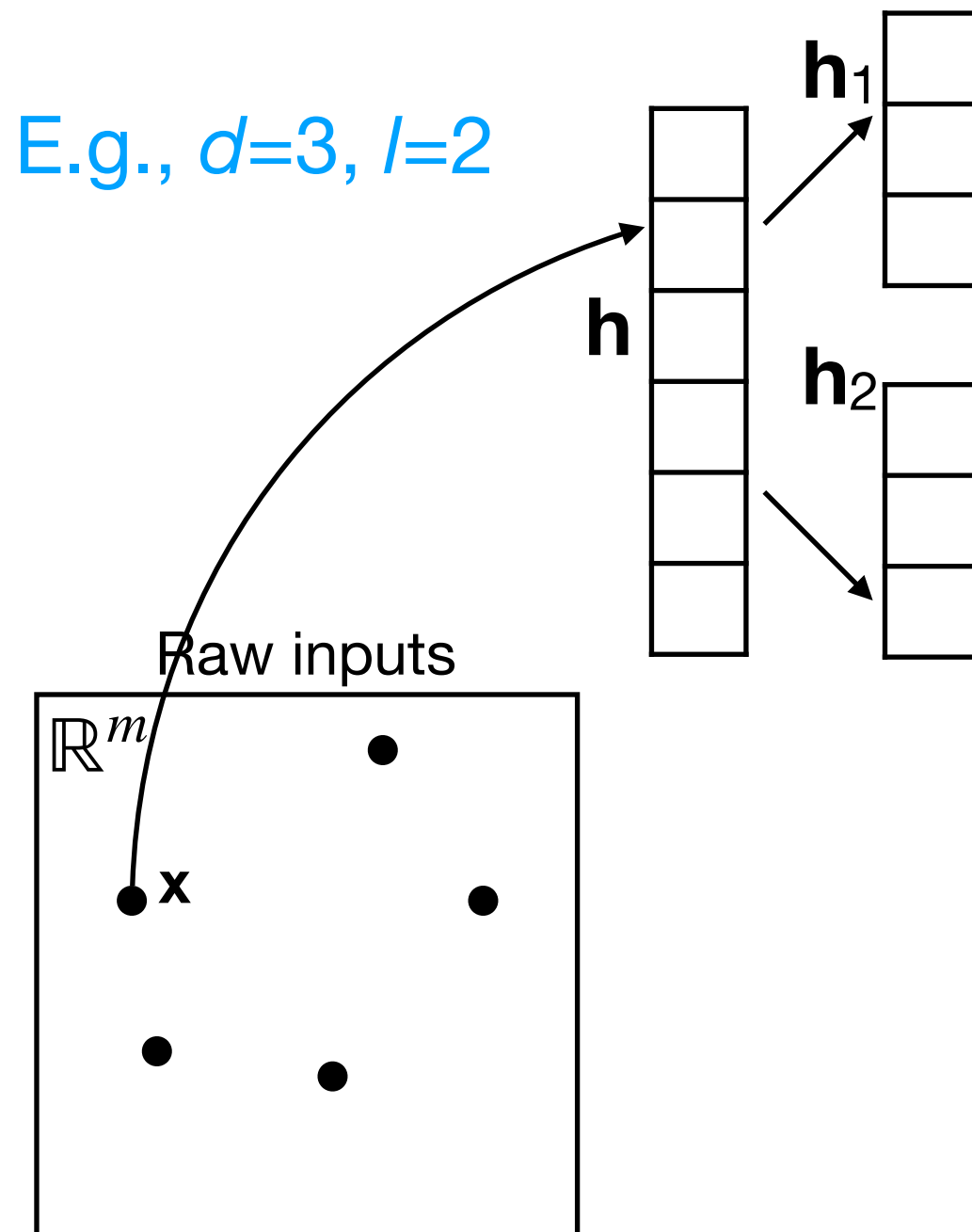
- Here's how we actually construct a deep VQ-VAE:
  1. Transform each  $\mathbf{x} \in \mathbb{R}^m$  into a feature vector  $\mathbf{h} \in \mathbb{R}^{l \times d}$ .  
I.e., / different pieces.

E.g.,  $d=3, l=2$



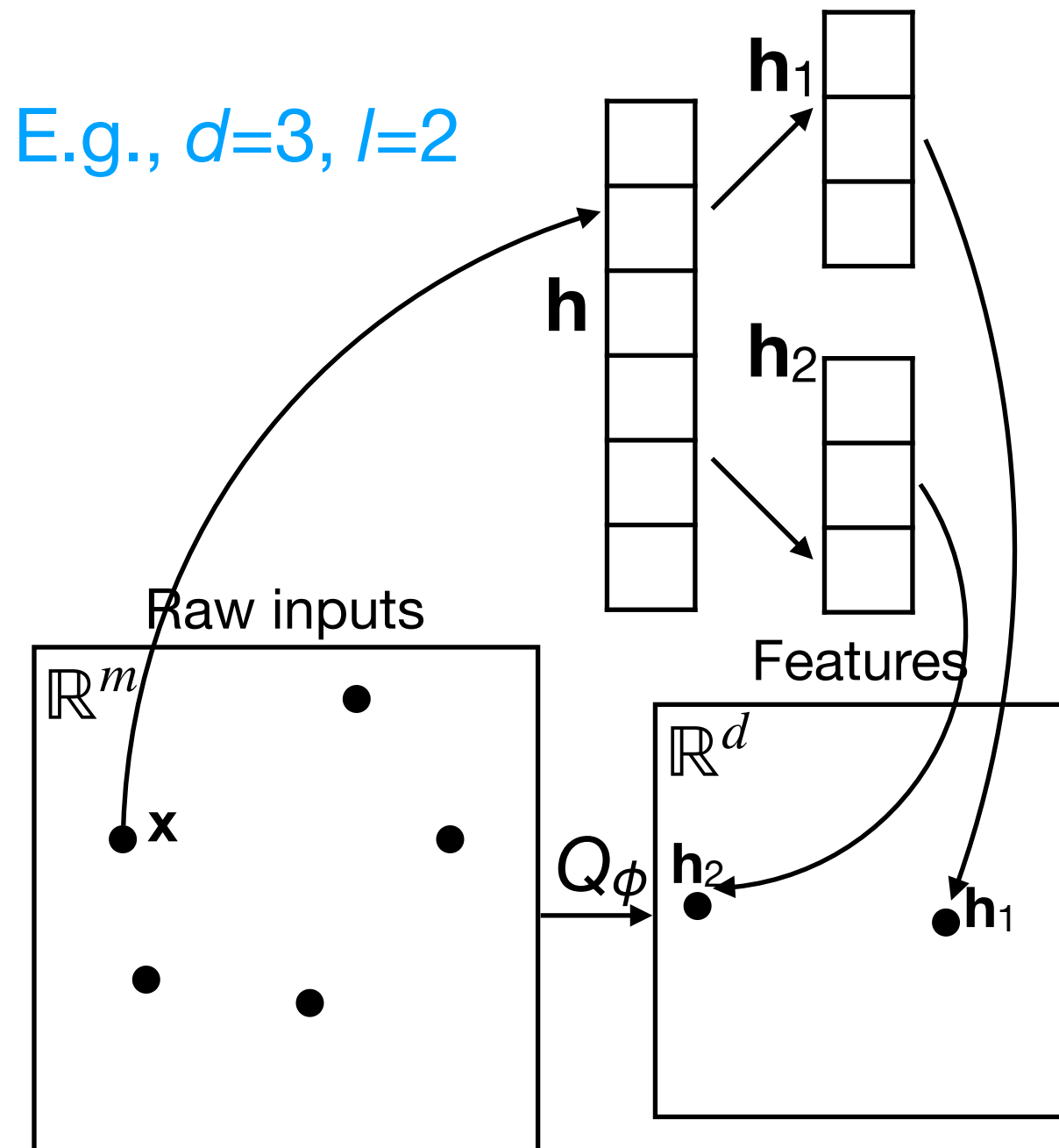
# (Deep) VQ-VAEs

- Here's how we actually construct a deep VQ-VAE:
  - Transform each  $\mathbf{x} \in \mathbb{R}^m$  into a feature vector  $\mathbf{h} \in \mathbb{R}^{l \times d}$ .
  - Split  $\mathbf{h}$  into multiple ( $l$ ) vectors  $\mathbf{h}_1, \dots, \mathbf{h}_l$ .



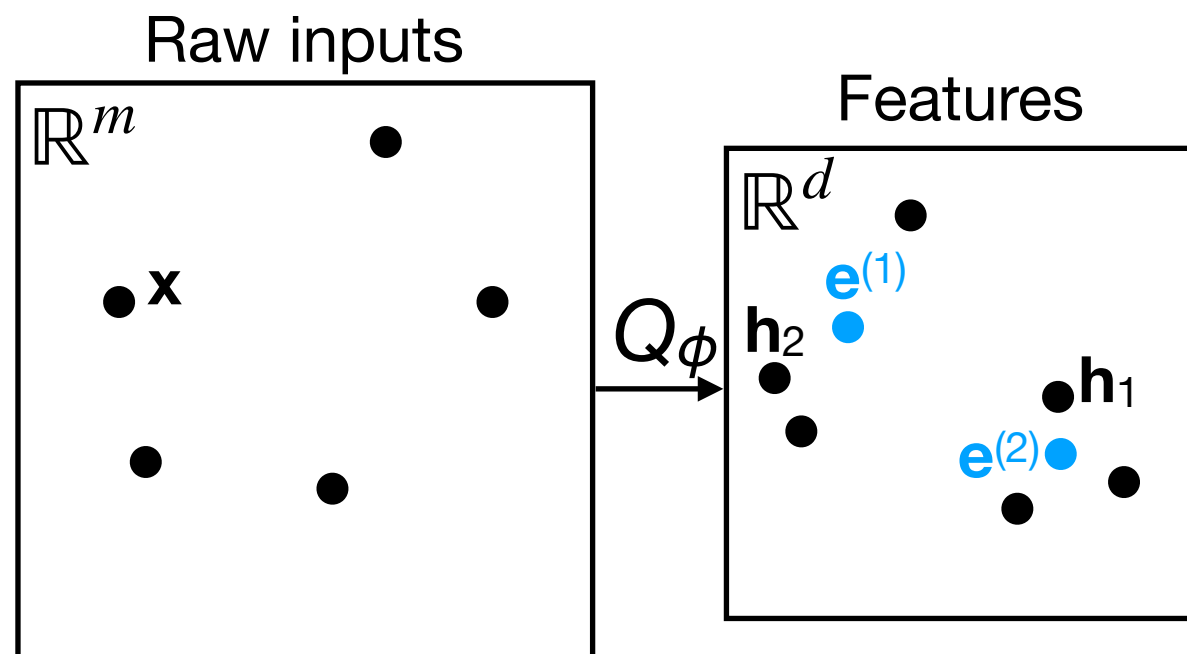
# (Deep) VQ-VAEs

- Here's how we actually construct a deep VQ-VAE:
  - Transform each  $\mathbf{x} \in \mathbb{R}^m$  into a feature vector  $\mathbf{h} \in \mathbb{R}^{l \times d}$ .
  - Split  $\mathbf{h}$  into multiple ( $l$ ) vectors  $\mathbf{h}_1, \dots, \mathbf{h}_l$ .



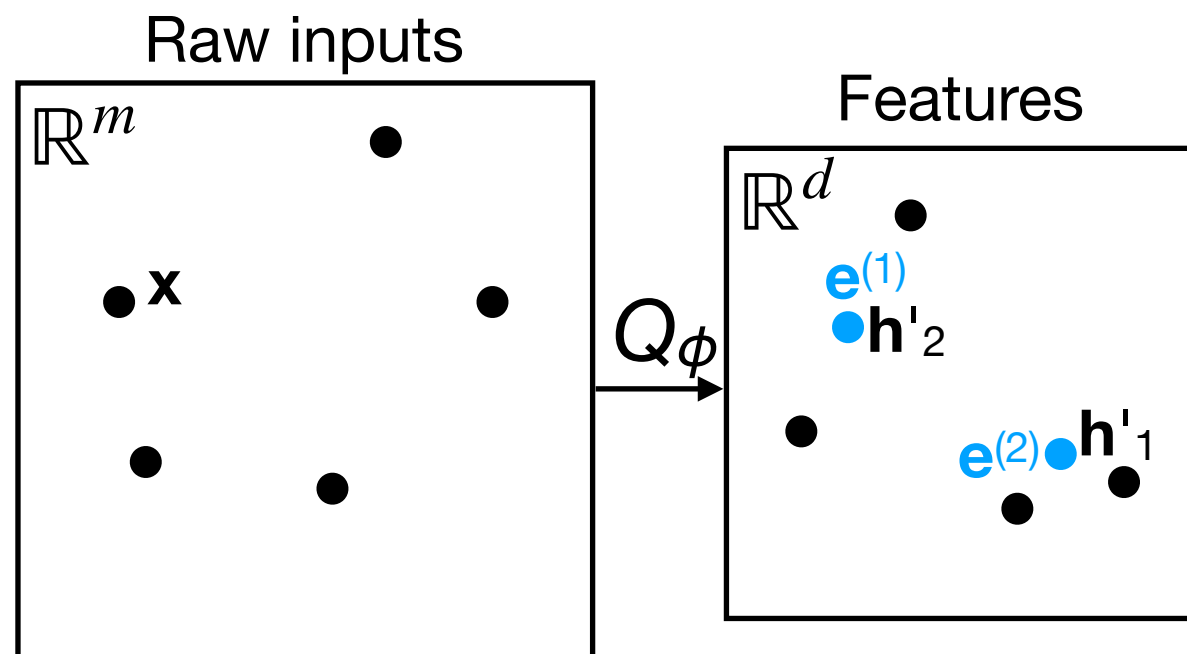
# (Deep) VQ-VAEs

- Here's how we actually construct a deep VQ-VAE:
  1. Transform each  $\mathbf{x} \in \mathbb{R}^m$  into a feature vector  $\mathbf{h} \in \mathbb{R}^{l \times d}$ .
  2. Split  $\mathbf{h}$  into multiple ( $l$ ) vectors  $\mathbf{h}_1, \dots, \mathbf{h}_l$ .
  3. Using running estimates of  $K$  cluster centroids over  $\{\mathbf{h}_j^{(i)}\}_{i,j}$ ,



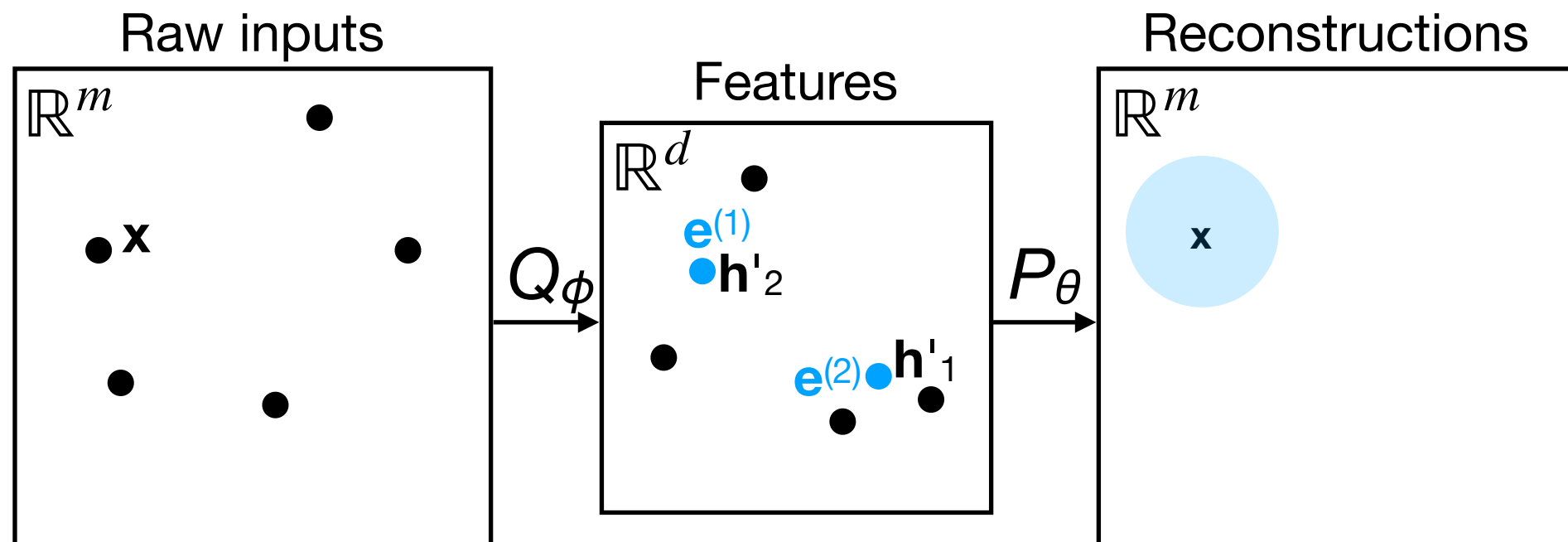
# (Deep) VQ-VAEs

- Here's how we actually construct a deep VQ-VAE:
  1. Transform each  $\mathbf{x} \in \mathbb{R}^m$  into a feature vector  $\mathbf{h} \in \mathbb{R}^{l \times d}$ .
  2. Split  $\mathbf{h}$  into multiple ( $l$ ) vectors  $\mathbf{h}_1, \dots, \mathbf{h}_l$ .
  3. Using running estimates of  $K$  cluster centroids over  $\{\mathbf{h}_j^{(i)}\}_{i,j}$ , quantize each  $\mathbf{h}_j$  into  $\mathbf{h}'_j$  using the nearest centroid  $\mathbf{e}^{(z)}$ .



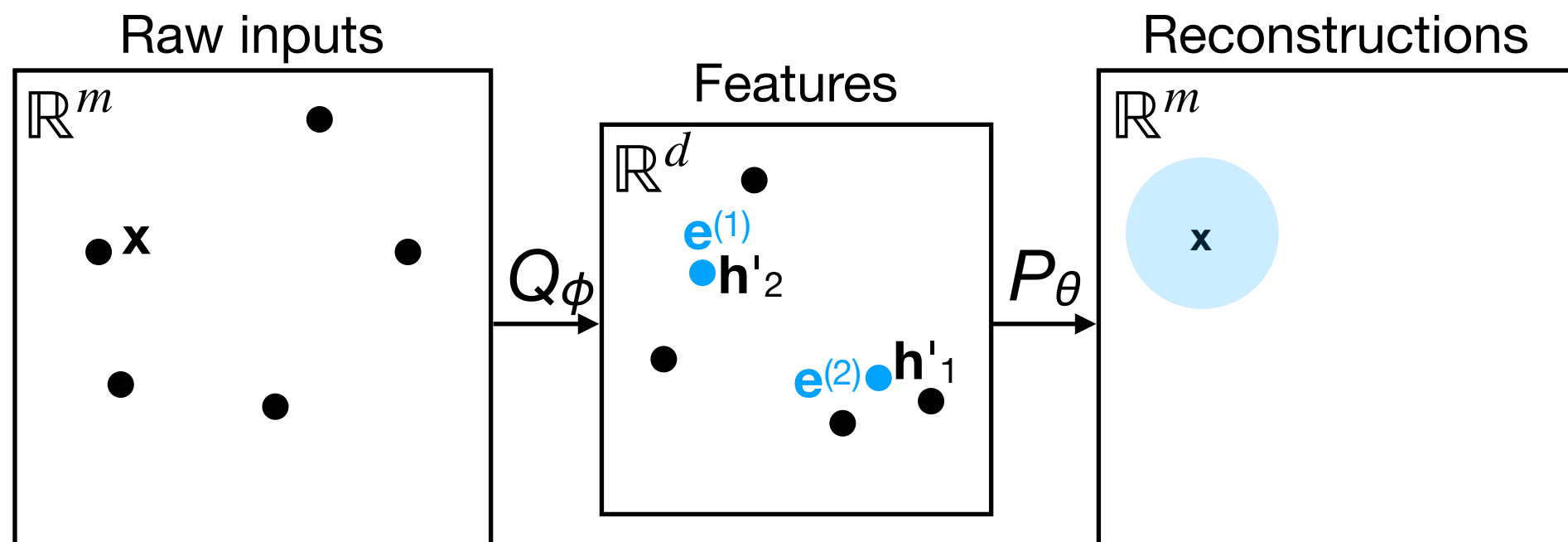
# (Deep) VQ-VAEs

- Here's how we actually construct a deep VQ-VAE:
  1. Transform each  $\mathbf{x} \in \mathbb{R}^m$  into a feature vector  $\mathbf{h} \in \mathbb{R}^{l \times d}$ .
  2. Split  $\mathbf{h}$  into multiple ( $l$ ) vectors  $\mathbf{h}_1, \dots, \mathbf{h}_l$ .
  3. Using running estimates of  $K$  cluster centroids over  $\{\mathbf{h}_j^{(i)}\}_{i,j}$ , quantize each  $\mathbf{h}_j$  into  $\mathbf{h}'_j$  using the nearest centroid  $\mathbf{e}^{(z)}$ .
  4. Concatenate the  $\mathbf{h}'_1, \dots, \mathbf{h}'_l$  into  $\mathbf{h}'$ , and then transform  $\mathbf{h}'$  into  $P(\mathbf{x} \mid \mathbf{h}') = P(\mathbf{x} \mid \{z_j\})$ .



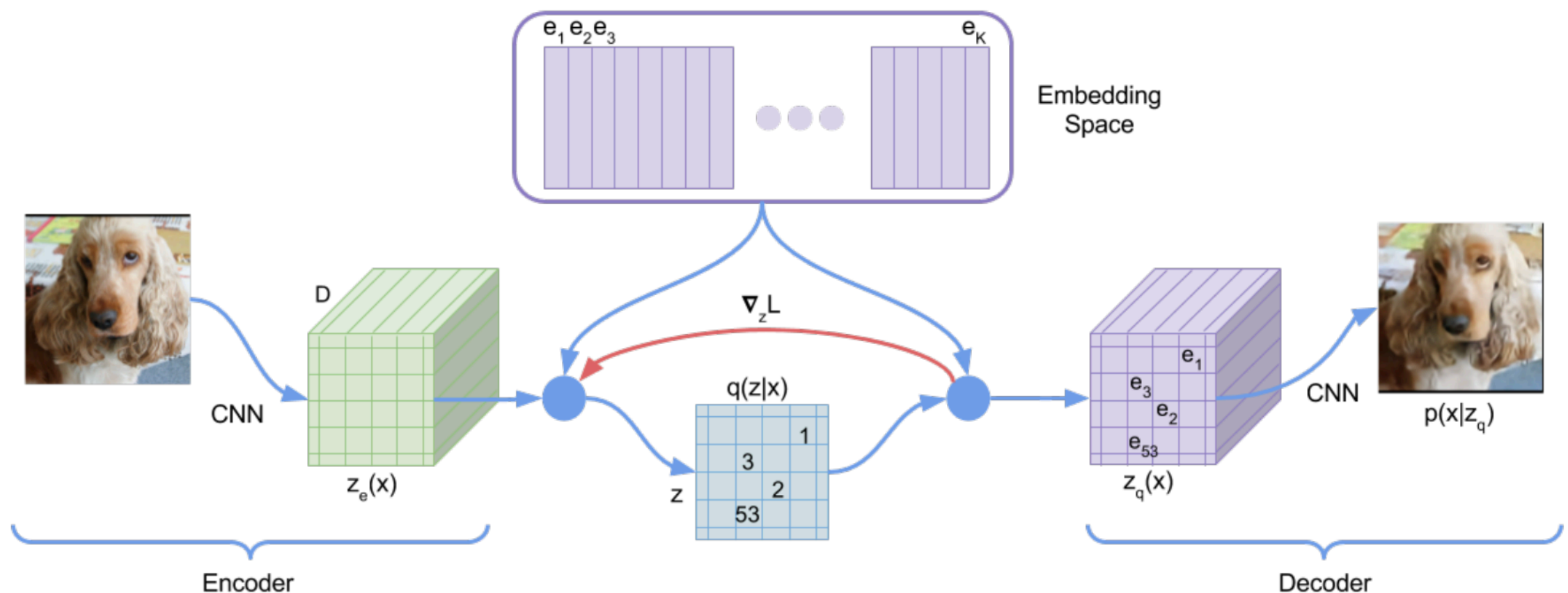
# (Deep) VQ-VAEs

- Now, each  $\mathbf{x}$  is represented by a **combination** of different points  $\mathbf{h}'$  in the latent space; this representation allows the decoder to reconstruct  $\mathbf{x}$  with high accuracy.
- This is much more powerful than  $K$ -means.



# (Deep) VQ-VAEs

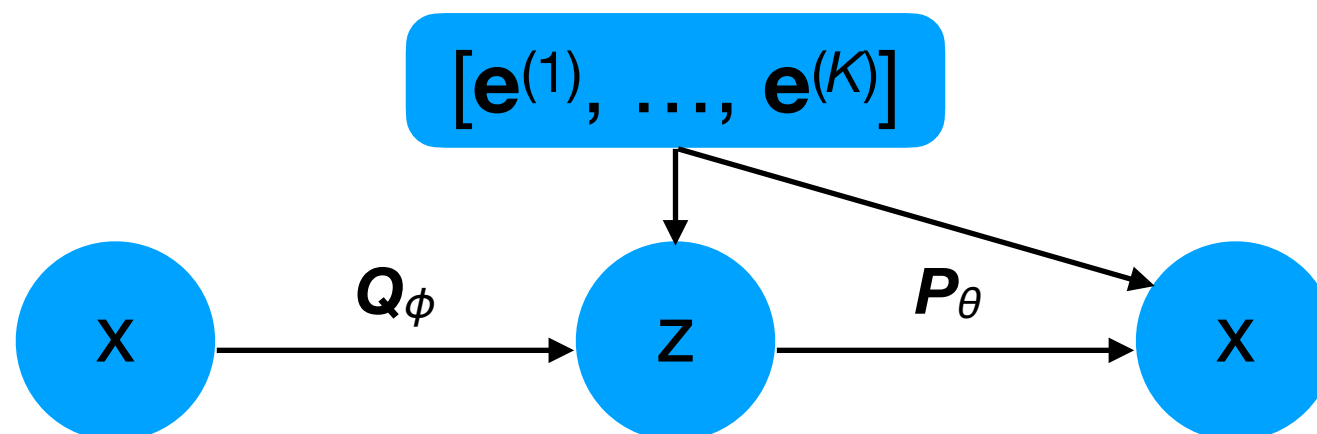
- Now, each  $\mathbf{x}$  is represented by a **combination** of different points  $\mathbf{h}'$  in the latent space; this representation allows the decoder to reconstruct  $\mathbf{x}$  with high accuracy.
- This is much more powerful than  $K$ -means.



# (Deep) VQ-VAEs

- The parameters that must be learned in this VQ-VAE include  $\phi$  (encoder),  $\theta$  (decoder), and  $\mathbf{E}=[\mathbf{e}^{(1)}, \dots, \mathbf{e}^{(K)}]$  (cluster centroids).
- To optimize these parameters, we will start with an MLE approach but then modify it as necessary.
- Recall the ELBO for continuous VAEs:

$$-D_{\text{KL}}(Q_{\phi}(z \mid \mathbf{x}) \mid P(z)) + \mathbb{E}_{Q_{\phi}}[\log P(\mathbf{x} \mid z)]$$



# $D_{\text{KL}}$ for VQ-VAEs

- For VQ-VAEs, we want  $P(z) = \mathcal{U}(1, \dots, K) = \frac{1}{K} \forall z$ .

- We typically use the encoder:

$$Q(z \mid \mathbf{x}) = \begin{cases} 1 & \text{if } z = \arg \min_k \|\mathbf{x} - \mathbf{e}^{(k)}\|^2 \\ 0 & \text{otherwise} \end{cases}$$

# $D_{\text{KL}}$ for VQ-VAEs

- For VQ-VAEs, we want  $P(z) = \mathcal{U}(1, \dots, K) = \frac{1}{K} \forall z$ .

- We typically use the encoder:

$$Q(z \mid \mathbf{x}) = \begin{cases} 1 & \text{if } z = \arg \min_k \|\mathbf{x} - \mathbf{e}^{(k)}\|^2 \\ 0 & \text{otherwise} \end{cases}$$

- By definition of KL divergence, we have:

$$D_{\text{KL}}(Q_\phi(z \mid \mathbf{x}) \mid P(z)) = \sum_{z=1}^K Q(z \mid \mathbf{x}) \log \frac{Q(z \mid \mathbf{x})}{P(z)}$$

*where  $0 \log 0$  is defined to be 0.*

# $D_{\text{KL}}$ for VQ-VAEs

- For VQ-VAEs, we want  $P(z) = \mathcal{U}(1, \dots, K) = \frac{1}{K} \forall z$ .

- We typically use the encoder:

$$Q(z \mid \mathbf{x}) = \begin{cases} 1 & \text{if } z = \arg \min_k \|\mathbf{x} - \mathbf{e}^{(k)}\|^2 \\ 0 & \text{otherwise} \end{cases}$$

- By definition of KL divergence, we have:

$$\begin{aligned} D_{\text{KL}}(Q_{\phi}(z \mid \mathbf{x}) \mid P(z)) &= \sum_{z=1}^K Q(z \mid \mathbf{x}) \log \frac{Q(z \mid \mathbf{x})}{P(z)} \\ &= 1 \log \frac{1}{\frac{1}{K}} + \sum_{\dots} 0 \log \frac{0}{\frac{1}{K}} \\ &= \log K \end{aligned}$$

# $D_{\text{KL}}$ for VQ-VAEs

- Since  $\log K$  does not depend on any of the VQ-VAE's parameters  $(\phi, \theta, \mathbf{E})$ , it can be ignored from the ELBO.
- That just leaves:

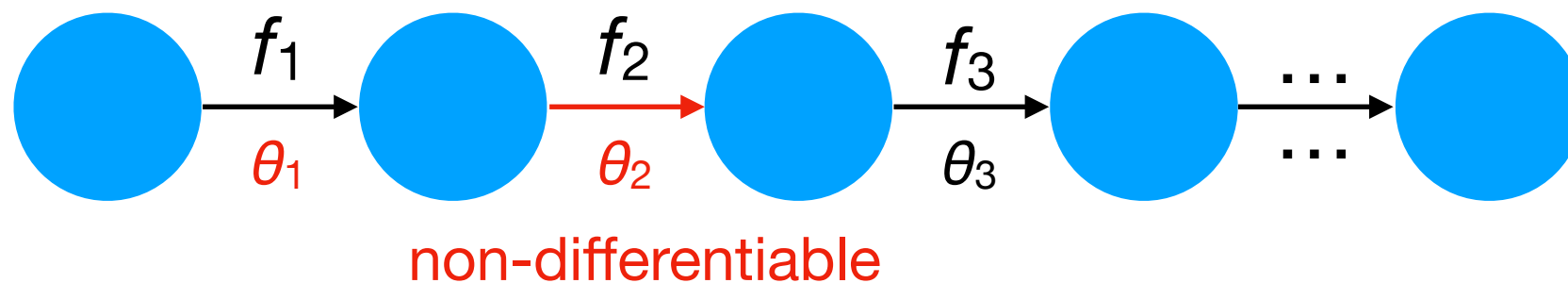
$$-\cancel{D_{\text{KL}}(Q_{\phi}(z \mid \mathbf{x}) \mid P(z))} + \mathbb{E}_{Q_{\phi}}[\log P(\mathbf{x} \mid z)]$$

# $D_{\text{KL}}$ for VQ-VAEs

- In practice, this means that  $Q(z \mid \mathbf{x})$  and  $P(z)$  in VQ-VAEs tend to be very different from  $\mathcal{U}(1, \dots, K) = \frac{1}{K} \forall z$ .
- Hence, sampling from  $P(\mathbf{z})$  naively tends to produce very bad results.
- Instead, we may need to train an autoregressor on the latent space  $P(\mathbf{z})$ .

# Non-differentiability

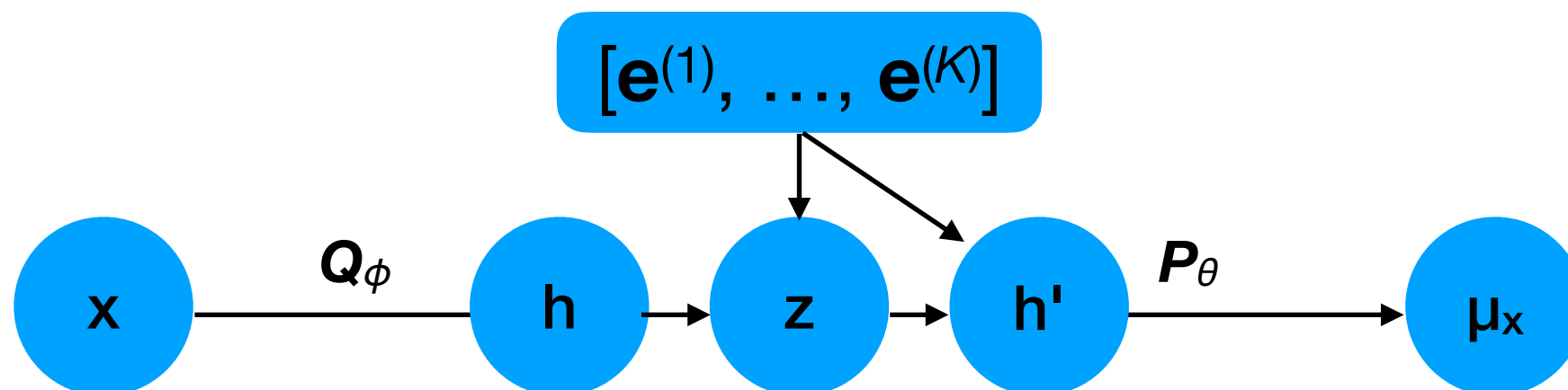
- In a computational graph (e.g., a NN), if any operation is non-differentiable, then we cannot use gradient descent to update any parameters in or before it.



# Reconstruction probability

$$-D_{\text{KL}}(Q_{\phi}(z \mid \mathbf{x}) \mid P(z)) + \mathbb{E}_{Q_{\phi}}[\log P(\mathbf{x} \mid z)]$$

- Like with a VAE, we could try to estimate the second term in the ELBO by sampling  $K=1$  samples, i.e.:
  1. Sample  $z$  from  $Q(z \mid \mathbf{x})$ .
  2. Compute  $\log P(\mathbf{x} \mid z)$ , e.g.,  $-\|\mathbf{x} - \mu_{\mathbf{x}}\|^2$



# Reconstruction probability

$$-D_{\text{KL}}(Q_{\phi}(z \mid \mathbf{x}) \mid P(z)) + \mathbb{E}_{Q_{\phi}}[\log P(\mathbf{x} \mid z)]$$

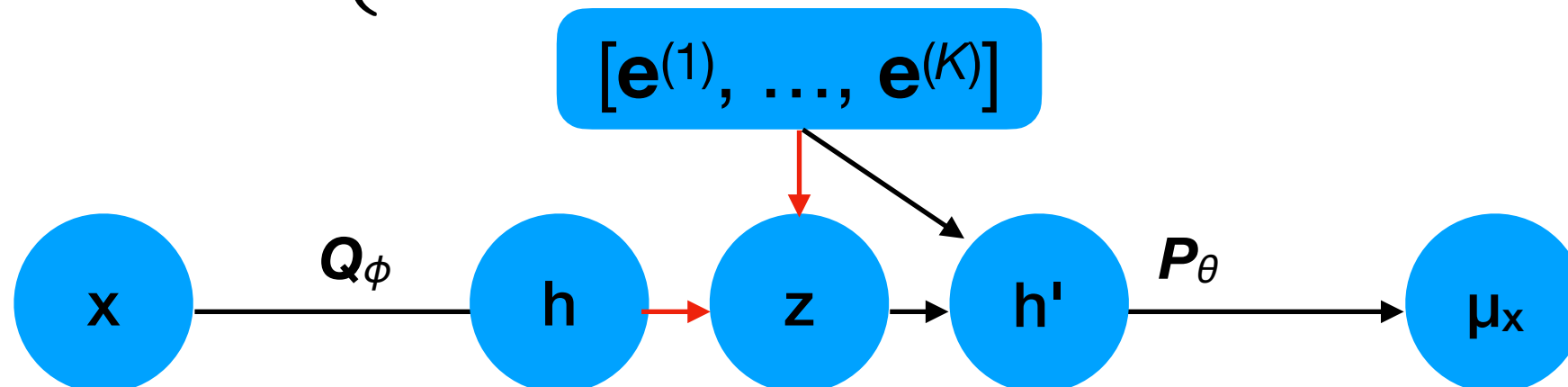
- Like with a VAE, we could try to estimate the second term in the ELBO by sampling  $K=1$  samples, i.e.:

1. Sample  $z$  from  $Q(z \mid \mathbf{x})$ .

2. Compute  $\log P(\mathbf{x} \mid z)$ , e.g.,  $-\|\mathbf{x} - \mu_{\mathbf{x}}\|^2$

- Like with continuous VAE, sampling is non-differentiable:

$$Q(z \mid \mathbf{x}) = \begin{cases} 1 & \text{if } z = \arg \min_k \|\mathbf{x} - \mathbf{e}^{(k)}\|^2 \\ 0 & \text{otherwise} \end{cases}$$



# Reconstruction probability

$$-D_{\text{KL}}(Q_{\phi}(z \mid \mathbf{x}) \mid P(z)) + \mathbb{E}_{Q_{\phi}}[\log P(\mathbf{x} \mid z)]$$

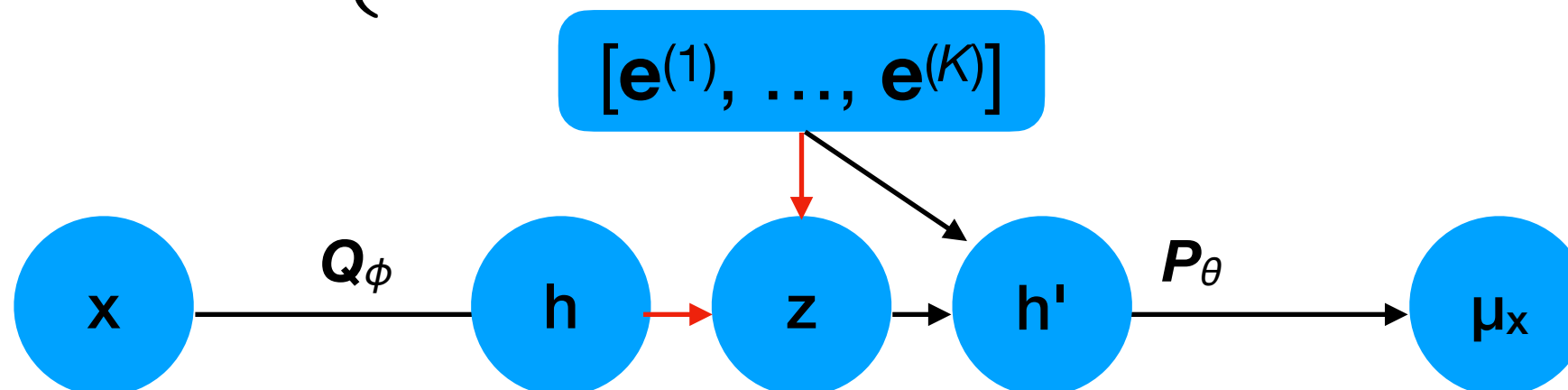
- Like with a VAE, we could try to estimate the second term in the ELBO by sampling  $K=1$  samples, i.e.:

1. Sample  $z$  from  $Q(z \mid \mathbf{x})$ .

2. Compute  $\log P(\mathbf{x} \mid z)$ , e.g.,  $-\|\mathbf{x} - \mu_{\mathbf{x}}\|^2$

- Unlike continuous VAE, there is no simple reparam. trick\*:

$$Q(z \mid \mathbf{x}) = \begin{cases} 1 & \text{if } z = \arg \min_k \|\mathbf{x} - \mathbf{e}^{(k)}\|^2 \\ 0 & \text{otherwise} \end{cases}$$



\*The Gumbel softmax is sometimes used but is more complicated and not clearly better than the straight-through gradient estimator.

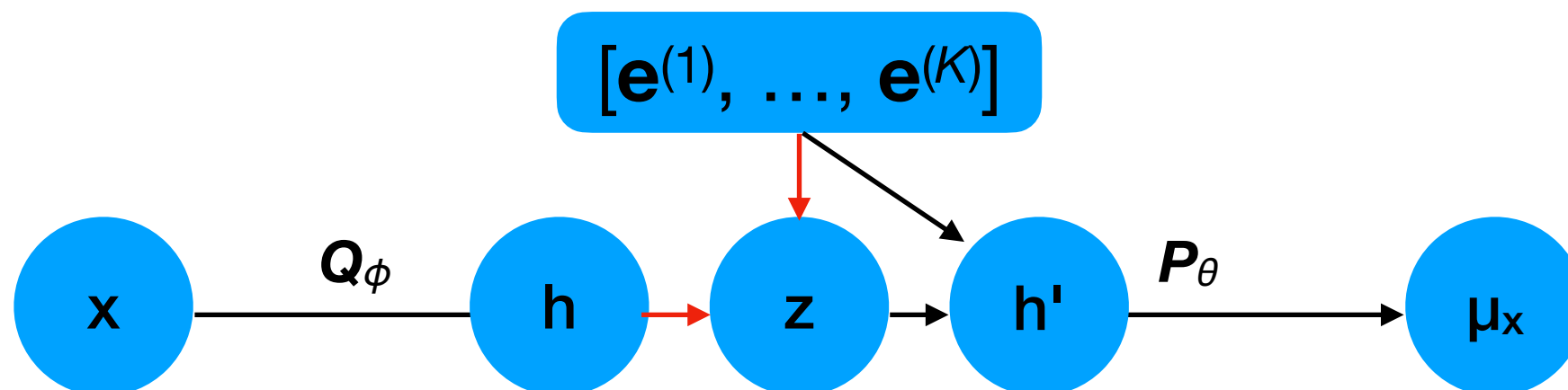
# Reconstruction probability

$$-D_{\text{KL}}(Q_{\phi}(z \mid \mathbf{x}) \mid P(z)) + \mathbb{E}_{Q_{\phi}}[\log P(\mathbf{x} \mid z)]$$

- The concrete issue is that the path through the graph to  $\phi$  is blocked due to non-differentiability:

$$\frac{\partial \log P(\mathbf{x} \mid z)}{\partial \mathbf{h}'} \frac{\partial \mathbf{h}'}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \phi}$$

Why?



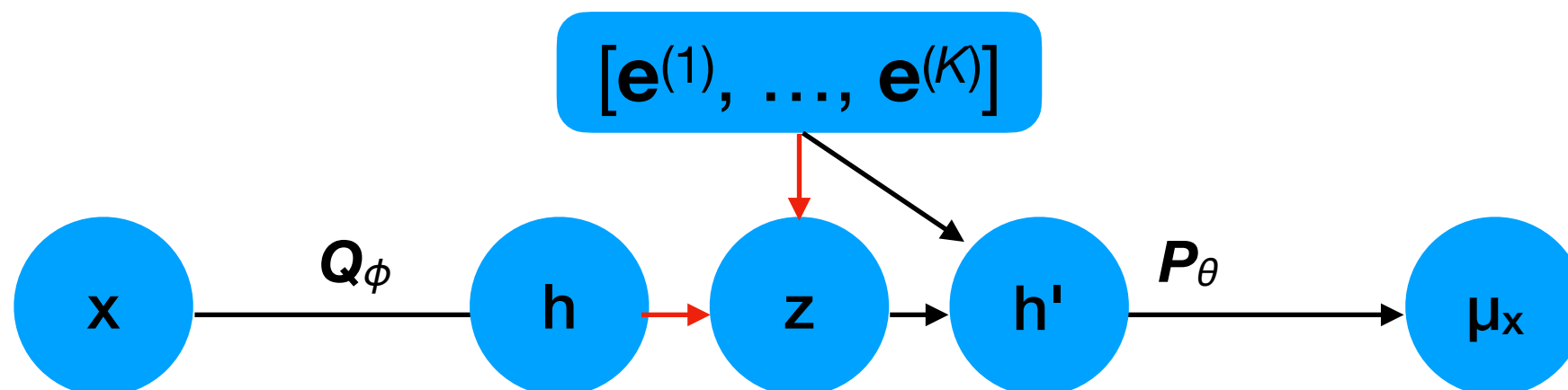
# Reconstruction probability

$$-D_{\text{KL}}(Q_{\phi}(z \mid \mathbf{x}) \mid P(z)) + \mathbb{E}_{Q_{\phi}}[\log P(\mathbf{x} \mid z)]$$

- The concrete issue is that the path through the graph to  $\phi$  is blocked due to non-differentiability:

$$\frac{\partial \log P(\mathbf{x} \mid z)}{\partial \mathbf{h}'} \frac{\partial \mathbf{h}'}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \phi}$$

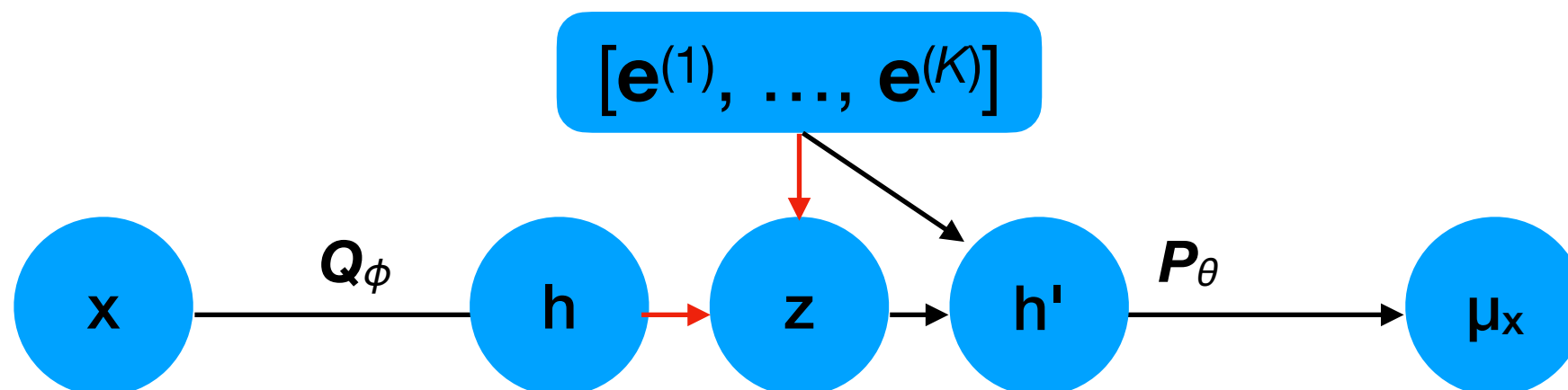
Reason: A small change in  $\mathbf{h}$  either causes *no change* in  $\mathbf{h}'$  or a *discontinuous jump* in  $\mathbf{h}'$ .



# VQVAE: Loss function

- For this reason, we have to “give up” on a first-principles approach to maximizing  $\mathbb{E}_{Q_\phi}[P(\mathbf{x} \mid z)]$  for VQ-VAEs.
- Instead, we hand-design a loss function with the goals:
  1. Encourage each  $\mathbf{h}^{(i)}$  to give a high  $\log P(\mathbf{x} \mid z)$ .

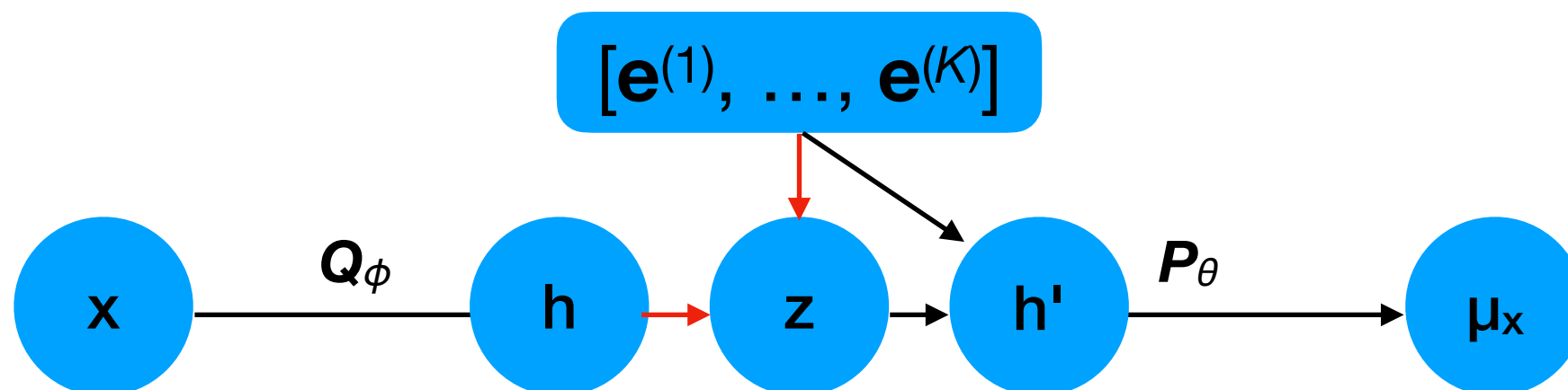
$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2$$



# VQVAE: Loss function

- For this reason, we have to “give up” on a first-principles approach to maximizing  $\mathbb{E}_{Q_\phi}[P(\mathbf{x} \mid z)]$  for VQ-VAEs.
- Instead, we hand-design a loss function with the goals:
  1. Encourage each  $\mathbf{h}^{(i)}$  to give a high  $\log P(\mathbf{x} \mid z)$ .
  2. Encourage the  $\mathbf{e}^{(1)}, \dots, \mathbf{e}^{(K)}$  to be the centroids of  $K$  clusters among the  $\{\mathbf{h}_j^{(i)}\}_{i,j}$ . *Pick centroids to represent the data.*

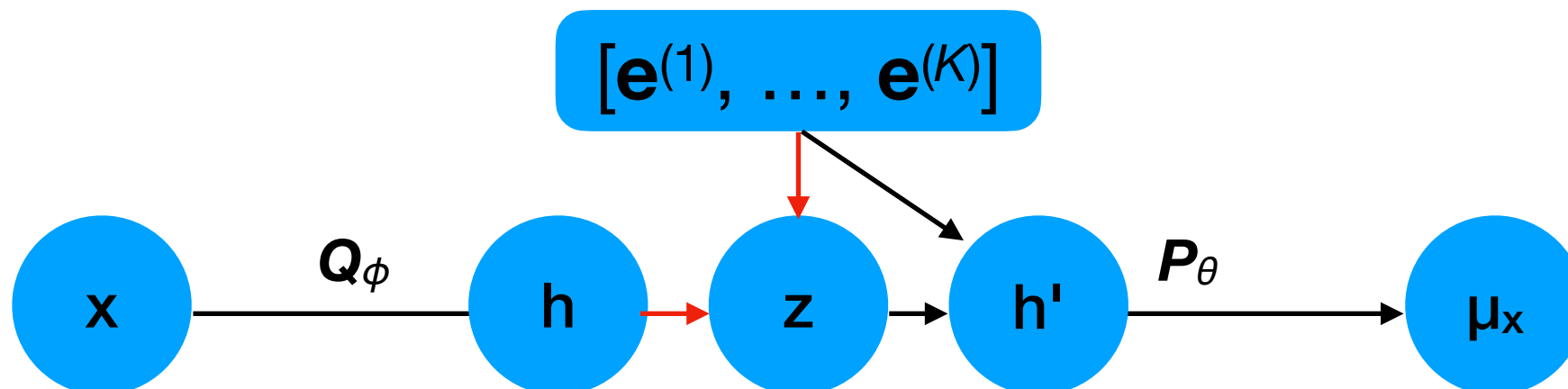
$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2 + \|\mathbf{h} - \mathbf{e}^{(z)}\|^2$$



# VQVAE: Loss function

- For this reason, we have to “give up” on a first-principles approach to maximizing  $\mathbb{E}_{Q_\phi}[P(\mathbf{x} \mid z)]$  for VQ-VAEs.
- Instead, we hand-design a loss function with the goals:
  1. Encourage each  $\mathbf{h}^{(i)}$  to give a high  $\log P(\mathbf{x} \mid z)$ .
  2. Encourage the  $\mathbf{e}^{(1)}, \dots, \mathbf{e}^{(K)}$  to be the centroids of  $K$  clusters among the  $\{\mathbf{h}_j^{(i)}\}_{i,j}$ .  
Discourage  $\{\mathbf{h}^{(i)}\}$  from growing too large.
  3. Encourage each  $\mathbf{h}^{(i)}$  to be “close” to its nearest centroid  $\mathbf{e}^{(z)}$ .

$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2 + \|\mathbf{h} - \mathbf{e}^{(z)}\|^2$$

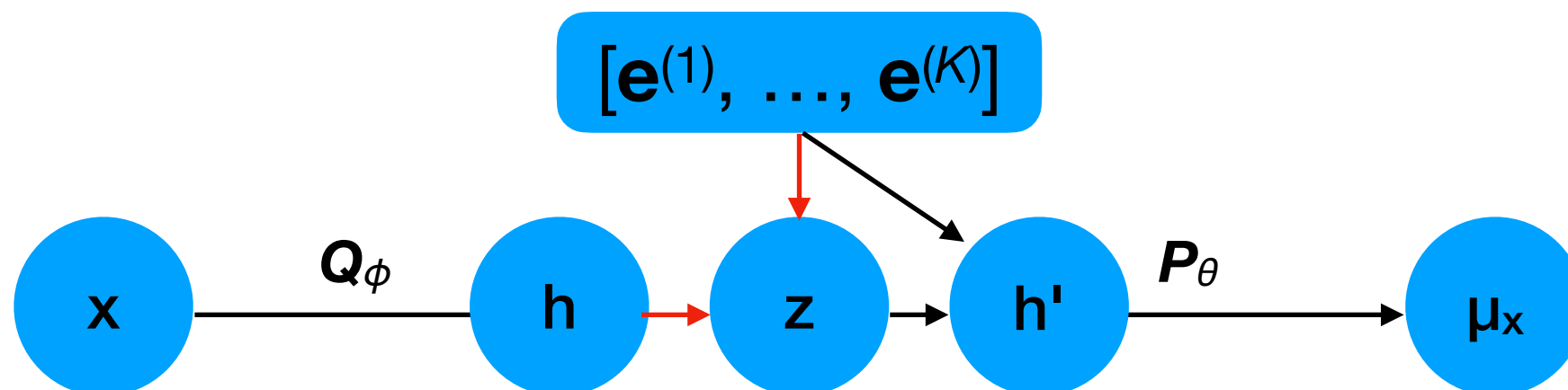


# VQVAE: Loss function

- We can express the gradient w.r.t.  $\phi$  as a product-of-Jacobians:

$$\frac{\partial \log P(\mathbf{x} \mid z)}{\partial \mathbf{h}'} \frac{\partial \mathbf{h}'}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \phi}$$

$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2 + \|\mathbf{h} - \mathbf{e}^{(z)}\|^2$$



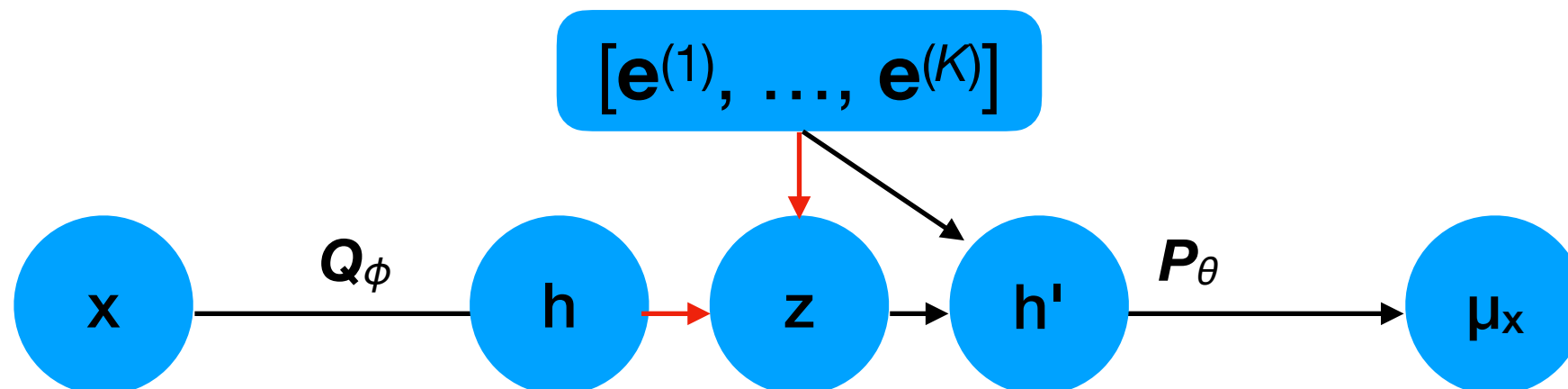
# VQVAE: Loss function

- We can express the gradient w.r.t.  $\phi$  as a product-of-Jacobians:

$$\frac{\partial \log P(\mathbf{x} \mid z)}{\partial \mathbf{h}'} \frac{\partial \mathbf{h}'}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \phi}$$

How a change in  $\mathbf{h}'$  affects reconstruction error

$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2 + \|\mathbf{h} - \mathbf{e}^{(z)}\|^2$$



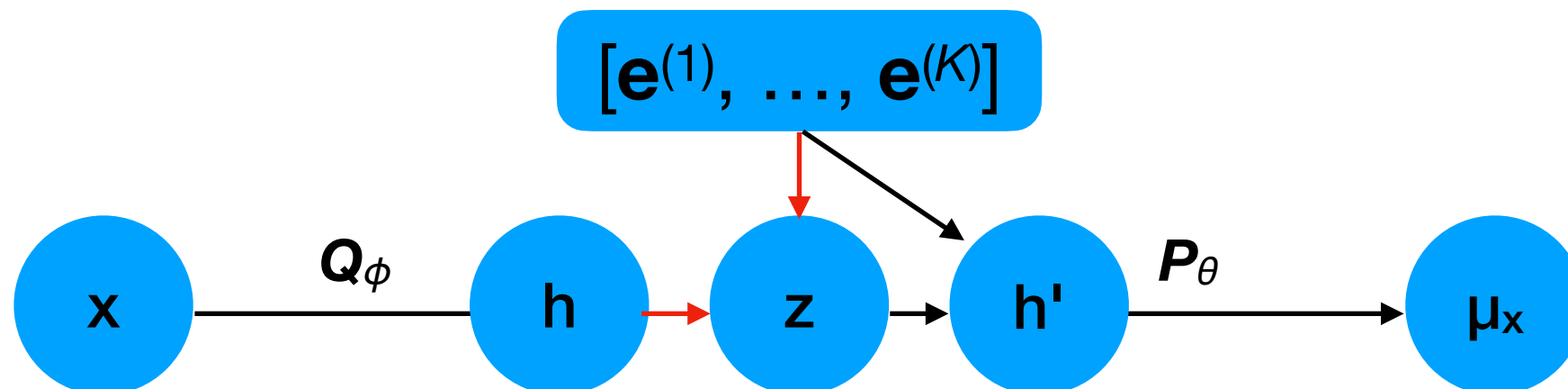
# VQVAE: Loss function

- We can express the gradient w.r.t.  $\phi$  as a product-of-Jacobians:

$$\frac{\partial \log P(\mathbf{x} \mid z)}{\partial \mathbf{h}'} \frac{\partial \mathbf{h}'}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \phi}$$

How a change in  $\mathbf{h}$   
affects  $\mathbf{h}'$

$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2 + \|\mathbf{h} - \mathbf{e}^{(z)}\|^2$$



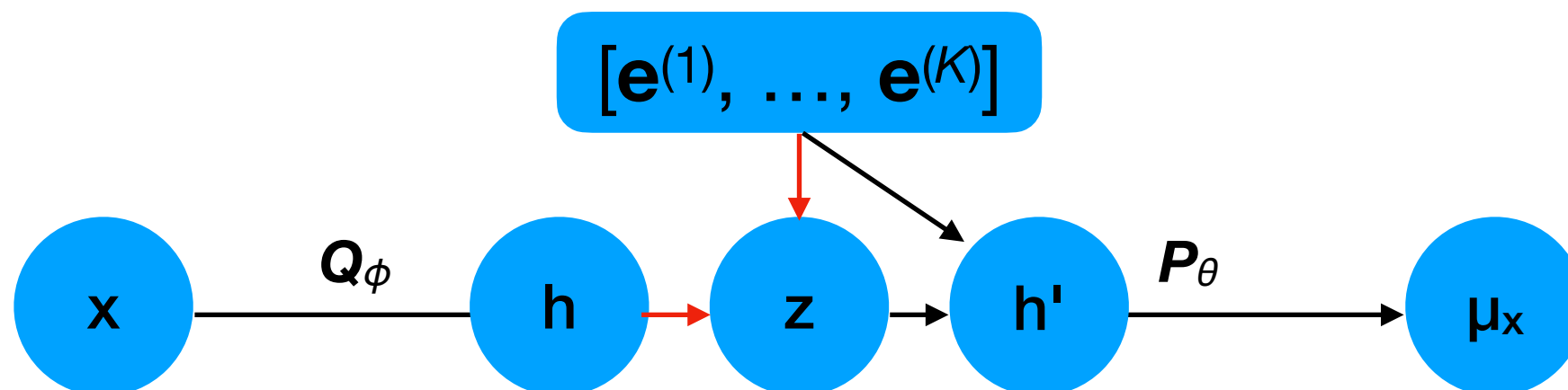
# VQVAE: Loss function

- We can express the gradient w.r.t.  $\phi$  as a product-of-Jacobians:

$$\frac{\partial \log P(\mathbf{x} \mid z)}{\partial \mathbf{h}'} \frac{\partial \mathbf{h}'}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \phi}$$

How the parameters  
of  $Q$  affect  $\mathbf{h}$

$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2 + \|\mathbf{h} - \mathbf{e}^{(z)}\|^2$$

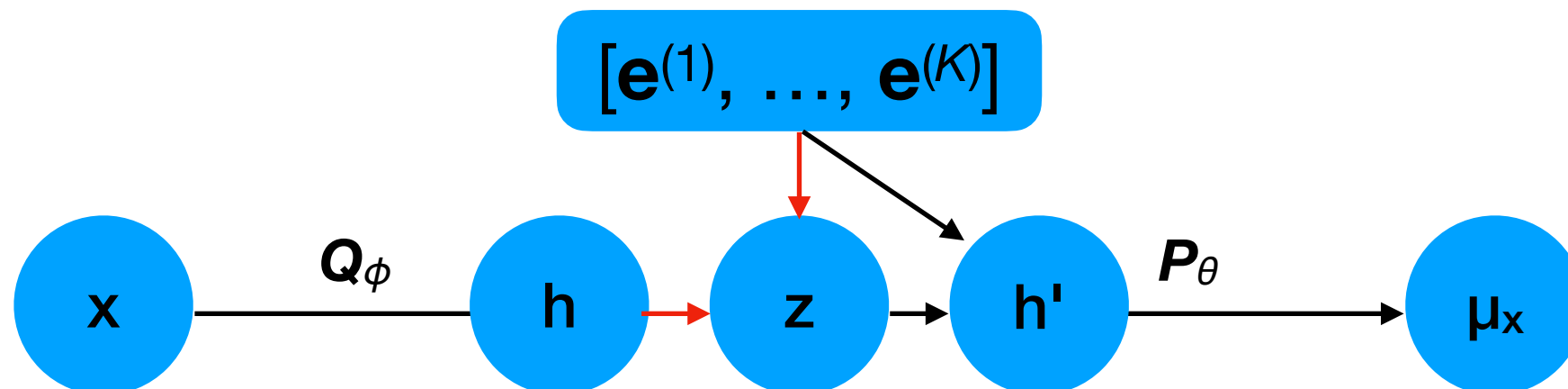


# VQVAE: Loss function

- But what about the non-differentiable map from  $\mathbf{h}$  to  $\mathbf{h}'$ ?

$$\frac{\partial \log P(\mathbf{x} \mid z)}{\partial \mathbf{h}'} \frac{\partial \mathbf{h}'}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \phi}$$

$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2 + \|\mathbf{h} - \mathbf{e}^{(z)}\|^2$$



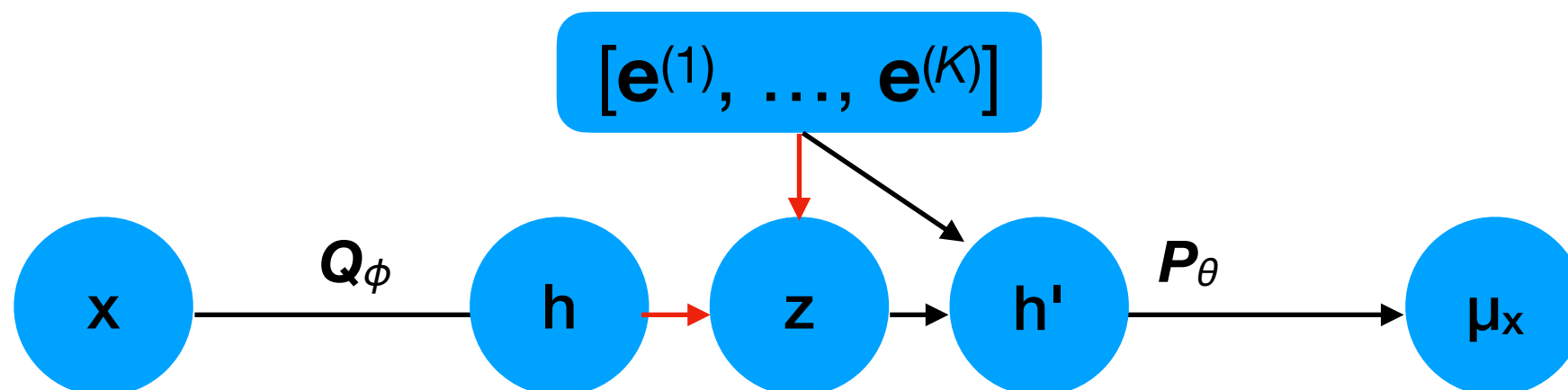
# VQVAE: Loss function

- But what about the non-differentiable map from  $\mathbf{h}$  to  $\mathbf{h}'$ ?

$$\frac{\partial \log P(\mathbf{x} \mid z)}{\partial \mathbf{h}'} \mathbf{I} \frac{\partial \mathbf{h}}{\partial \phi}$$

- Straight-through gradient estimator:** we just ignore it! I.e., we set it to  $\mathbf{I}$ .
- Why is this reasonable? My interpretation: a small change in  $\mathbf{h}$  *does* pull  $\mathbf{h}'$  toward  $\mathbf{h}$  (though scaled by inverse cluster size).

$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2 + \|\mathbf{h} - \mathbf{e}^{(z)}\|^2$$



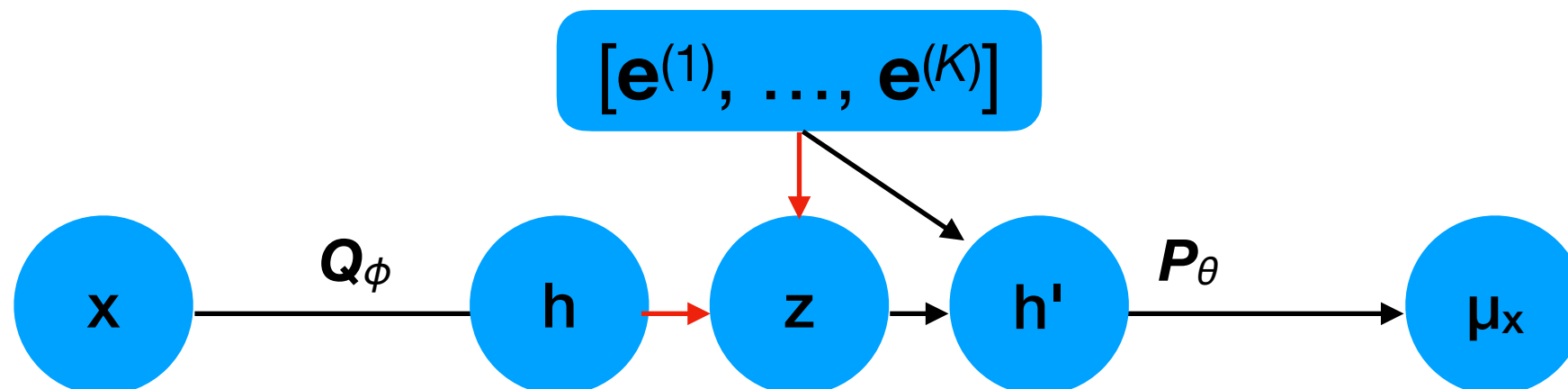
# VQVAE: Loss function

- Note that the second loss term has non-zero gradient w.r.t. both  $\mathbf{h}$  (the embedding of example  $\mathbf{x}$ ) and  $\mathbf{e}^{(z)}$  ( $\mathbf{x}$ 's assigned centroid):

$$\frac{\partial}{\partial \mathbf{h}} \|\mathbf{h} - \mathbf{e}^{(z)}\|^2 = 2(\mathbf{h} - \mathbf{e}^{(z)})$$

$$\frac{\partial}{\partial \mathbf{e}^{(z)}} \|\mathbf{h} - \mathbf{e}^{(z)}\|^2 = 2(\mathbf{e}^{(z)} - \mathbf{h})$$

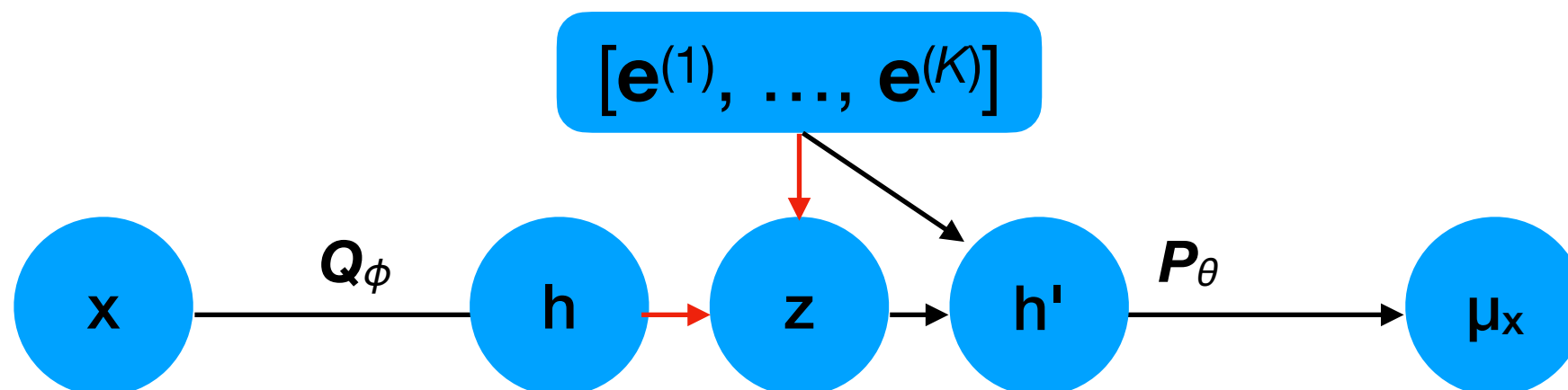
$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2 + \|\mathbf{h} - \mathbf{e}^{(z)}\|^2$$



# VQVAE: Loss function

- However, to get more flexibility in how we calculate and use these gradients, we typically split this term into 2 terms:
- Here, **sg** stands for “stop gradient” and prevents gradient for the variable in () from being calculated.

$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2 + \|\mathbf{h} - \text{sg}(\mathbf{e}^{(z)})\|^2 + \|\text{sg}(\mathbf{h}) - \mathbf{e}^{(z)}\|^2$$



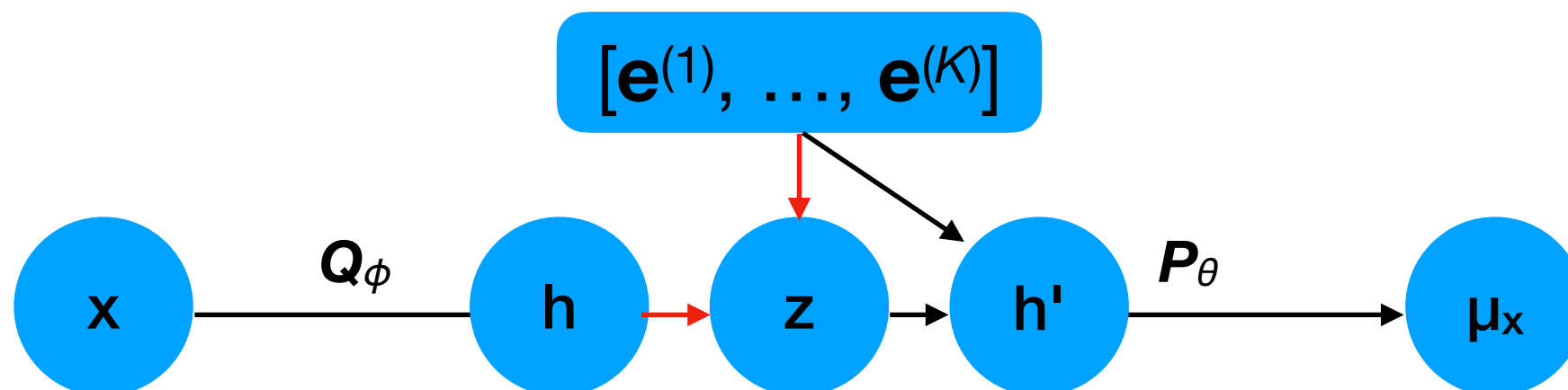
# VQVAE: Loss function

- Summing over both of the new terms, we still obtain the gradient expressions below.
- But now we can weight each loss term and do fancier things as well (e.g., exponential moving average).

$$\frac{\partial}{\partial \mathbf{h}} \|\mathbf{h} - \mathbf{e}^{(z)}\|^2 = 2(\mathbf{h} - \mathbf{e}^{(z)})$$

$$\frac{\partial}{\partial \mathbf{e}^{(z)}} \|\mathbf{h} - \mathbf{e}^{(z)}\|^2 = 2(\mathbf{e}^{(z)} - \mathbf{h})$$

$$f_{\text{loss}}(\phi, \theta, \mathbf{E}) = \|\mathbf{x} - \mu_{\mathbf{x}}\|^2 + \|\mathbf{h} - \text{sg}(\mathbf{e}^{(z)})\|^2 + \|\text{sg}(\mathbf{h}) - \mathbf{e}^{(z)}\|^2$$



# Generative models

## Shallow/Linear

## Deep/Non-linear

### Latent variable model (LVM)

Continuous

probabilistic PCA<sup>†</sup>

VAE<sup>†</sup>,  
diffusion, NF<sup>†</sup>

Discrete

$k$ -means,<sup>\*†</sup>  
GMM<sup>†</sup>

VQ-VAE<sup>†</sup>

### Autoregressive

Continuous

linear dynamical system,<sup>†</sup>  
AR( $p$ )<sup>\*†</sup>

seq2seq<sup>\*†</sup>

Discrete

conditional probability tables<sup>†</sup>

### Adversarial

GAN<sup>\*†</sup>

\* Squared-error / log-loss minimization

† Maximum likelihood estimation (MLE)

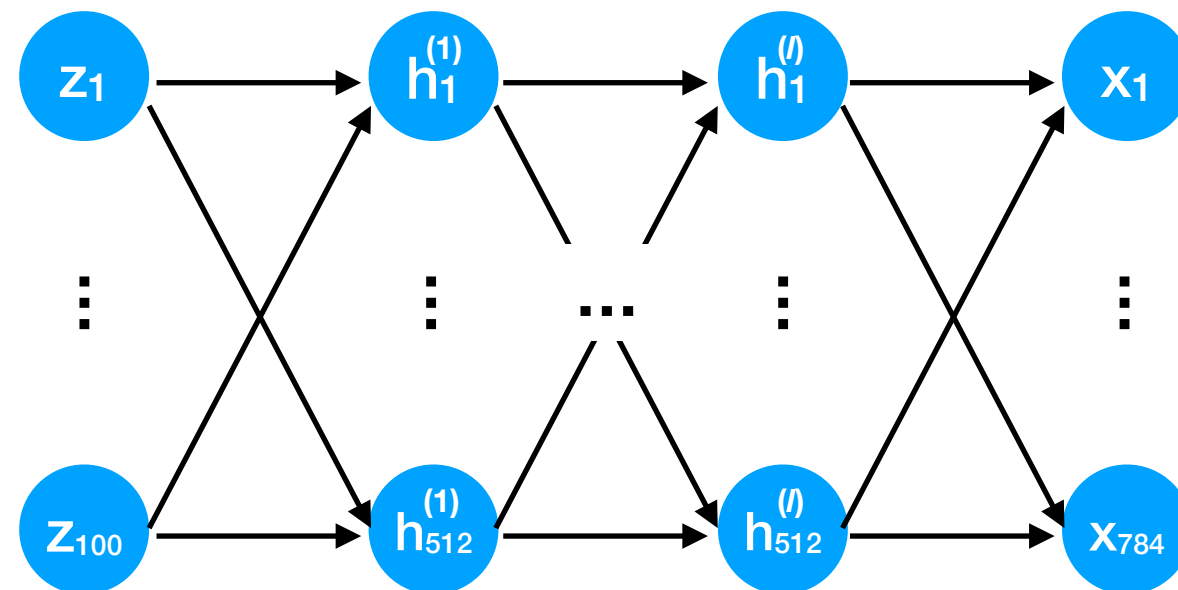
# Generative Adversarial Networks (GANs)

# Generative Adversarial Networks (GANs)

- Like VAEs, GANs consist of two components, but their semantics are different.
- Let  $P_{\text{data}}(\mathbf{x})$  be the ground-truth data distribution.
- **Generator  $G$** : given a noise vector  $\mathbf{z}$  from an easy-to-sample distribution (e.g., Gaussian, uniform), generate a vector  $\mathbf{x}$  that looks like it came from  $P_{\text{data}}(\mathbf{x})$ .
- **Discriminator  $D$** : given a vector  $\mathbf{x}$ , decide if it is real ( $\hat{y}=1$ ) or fake ( $\hat{y}=0$ ).  $D$  acts as a “forgery detector”.

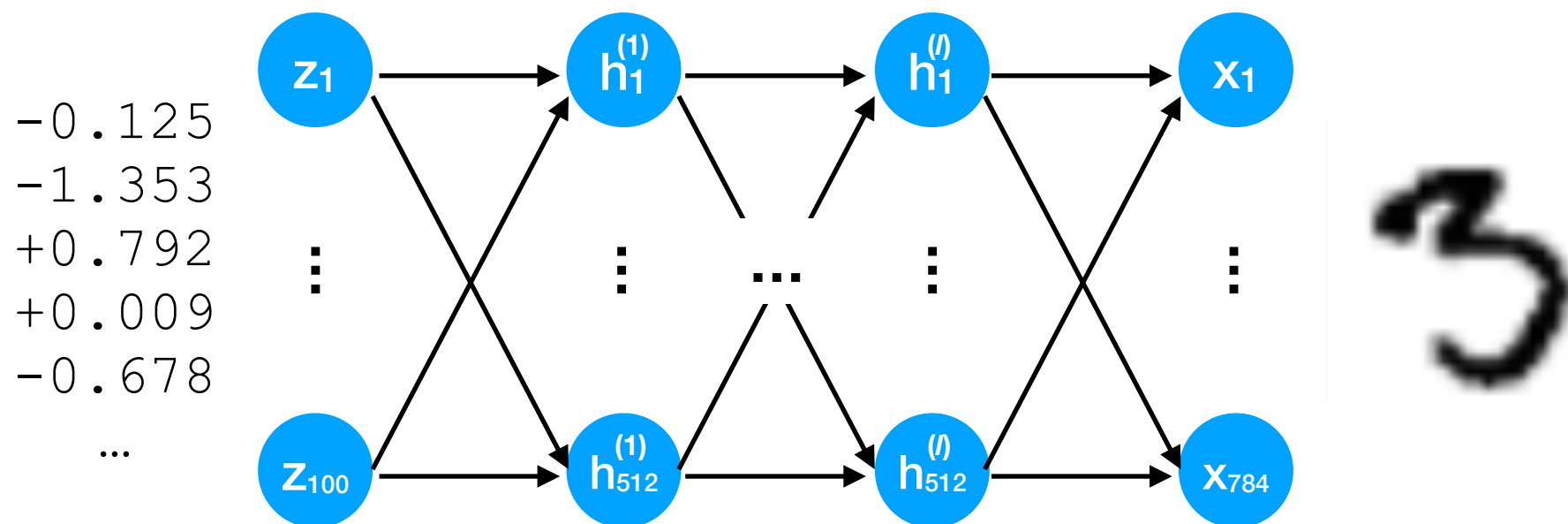
# Generator $G$

- Example  $G$  with  $l$  hidden layers that generates an MNIST image ( $28 \times 28 = 784$ )  $\mathbf{x}$  from a 100-dim noise vector  $\mathbf{z}$ :



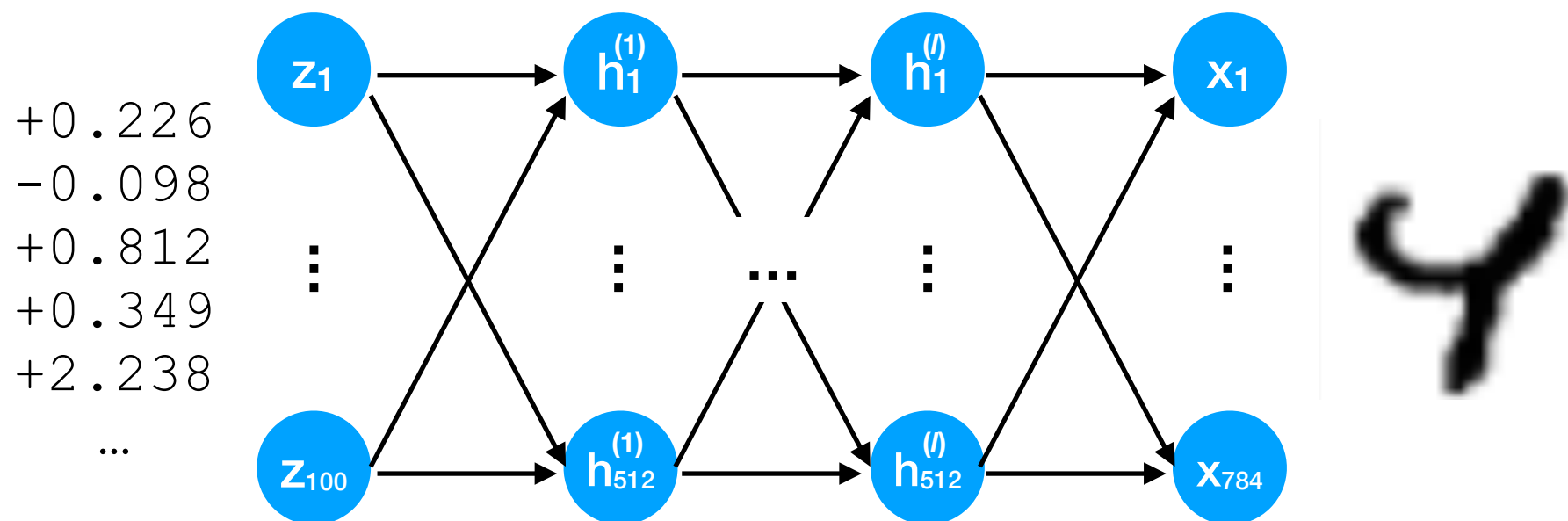
# Generator $G$

- By feeding different noise vectors  $\mathbf{z}$ , we obtain different  $\mathbf{x}$ .
- Implicitly,  $\mathbf{z}$  encodes the different dimensions of variability of  $\mathbf{X}$  (though they may not be intuitive or independent).



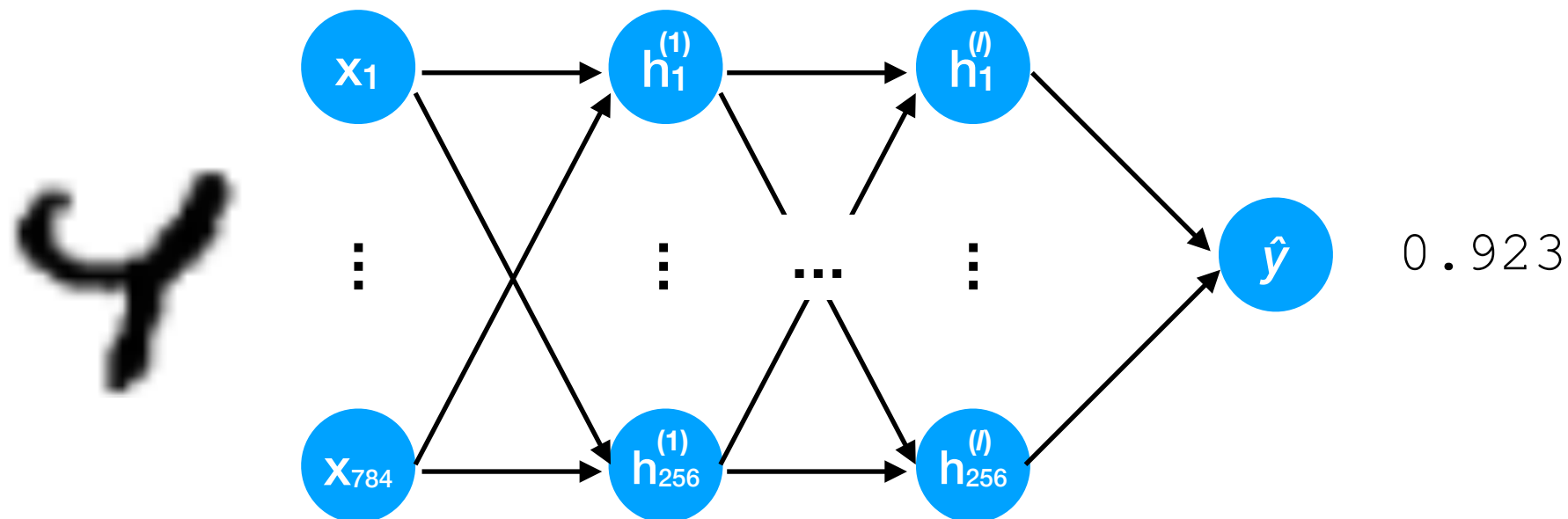
# Generator $G$

- By feeding different noise vectors  $\mathbf{z}$ , we obtain different  $\mathbf{x}$ .
- Implicitly,  $\mathbf{z}$  encodes the different dimensions of variability of  $\mathbf{X}$  (though they may not be intuitive or independent).



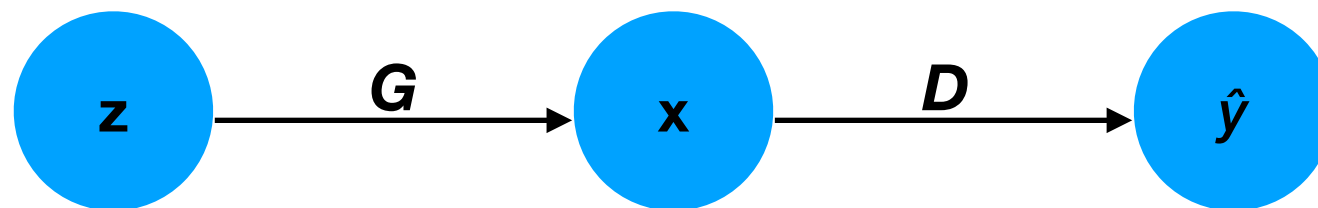
# Discriminator $D$

- Example  $D$  with  $l$  hidden layers that estimates  $\hat{y} \in (0,1)$  that expresses probability that the input  $\mathbf{x}$  is real:



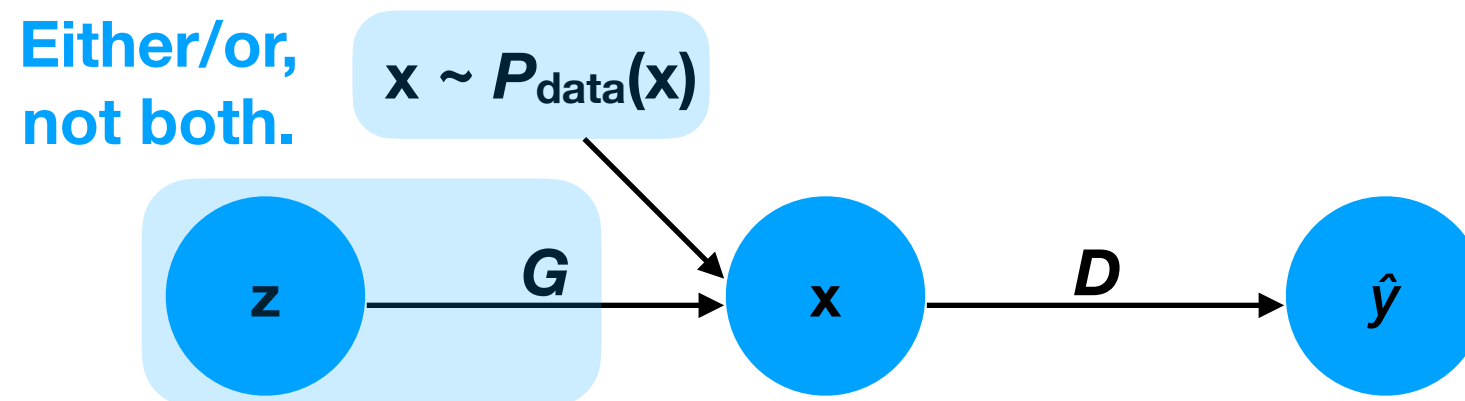
# Generative Adversarial Networks (GANs)

- Like VAEs, GANs are trained such that one component “feeds” to the other.



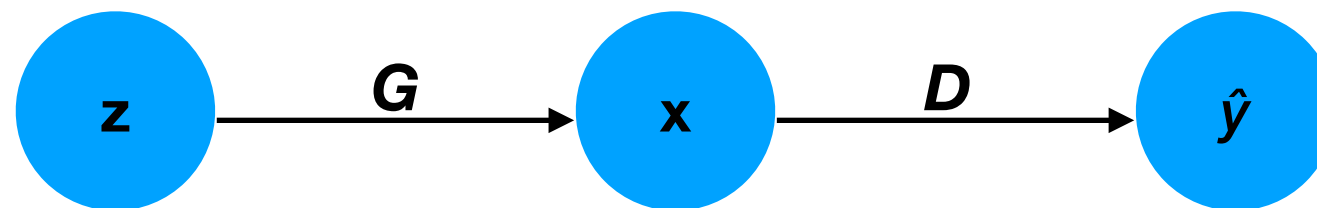
# Generative Adversarial Networks (GANs)

- Like VAEs, GANs are trained such that one component “feeds” to the other.
- In contrast to VAEs, the discriminator  $D$  is sometimes given a “fake” data vector  $\mathbf{x}$  (generated by  $G$ ), and sometimes given a “real” vector  $\mathbf{x}$  sampled from the training set (which approximates  $P_{\text{data}}(\mathbf{x})$ ).



# Generative Adversarial Networks (GANs)

- Each network has its own parameters:
  - $G$  has parameters  $\theta_G$ .
  - $D$  has parameters  $\theta_D$ .

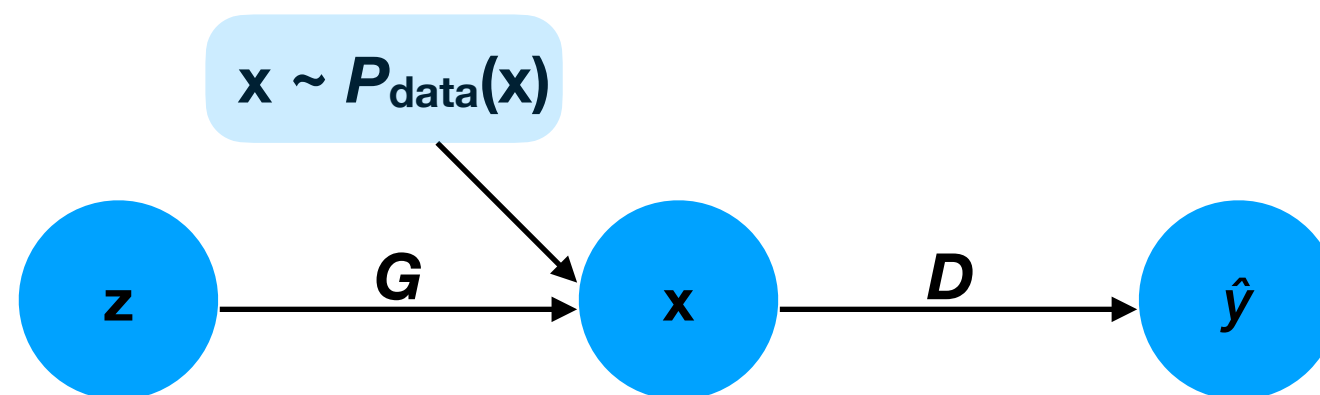


# Generative Adversarial Networks (GANs)

- We can define the following accuracy function of how well  $D$  can discriminate fake from real data:

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

**Exp. log-likelihood that  $D$  recognizes real data as real.**

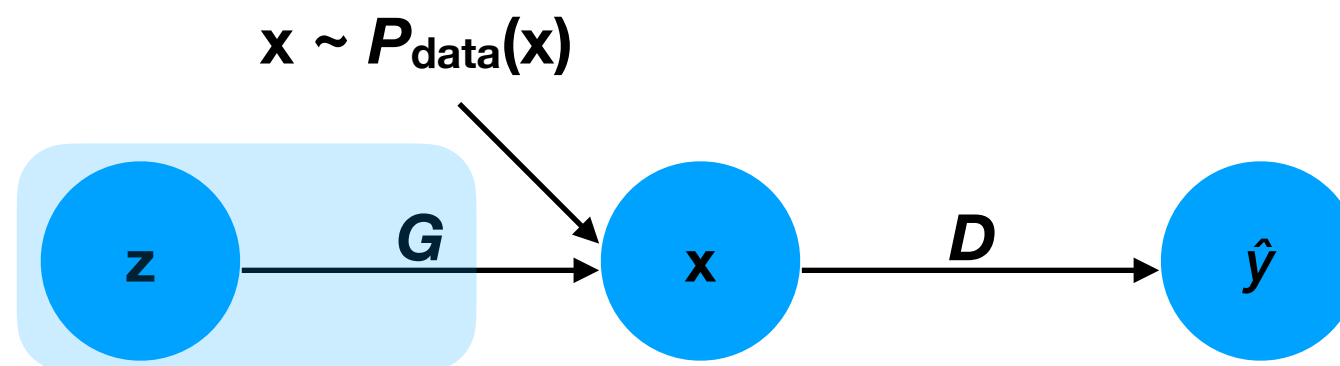


# Generative Adversarial Networks (GANs)

- We can define the following accuracy function of how well  $D$  can discriminate fake from real data:

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

Exp. log-likelihood that  $D$  recognizes fake data as fake.



# Generative Adversarial Networks (GANs)

- The goal of  $D$  is to *maximize*  $f_{\text{acc}}$ , whereas the goal of  $G$  is to *minimize*  $f_{\text{acc}}$ .

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

- $D$  and  $G$  compete against each other to find equilibrium:

$$\min_{\theta_G} \max_{\theta_D} f_{\text{acc}}(\theta_G, \theta_D)$$

- The equilibrium is  $D$  having 50% accuracy at detecting forgeries, and  $G$  generating  $\mathbf{x}$  according to  $P_{\text{data}}(\mathbf{x})$ .

# Training GANs

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

- Intuitively:
  - If  $D$  is a pretty good discriminator, that can help to figure out how to produce more realistic data  $\mathbf{x}$ .
  - If  $G$  is a pretty good generator, that can provide more and better training data to  $D$  to help distinguish real from fake data  $\mathbf{x}$ .

# Training GANs

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

- In practice, we train  $D$  and  $G$  *iteratively*:
  - Freeze  $G$ , and perform SGD on  $D$  for  $k$  iterations to increase  $f_{\text{acc}}$ .

# Training GANs

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

- In practice, we train  $D$  and  $G$  *iteratively*:
- Freeze  $G$ , and perform SGD on  $D$  for  $k$  iterations to increase  $f_{\text{acc}}$ .

**Improve  $D$ 's forgery detection accuracy  
for a fixed distribution of fake data.**

# Training GANs

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

- In practice, we train  $D$  and  $G$  *iteratively*:
  - Freeze  $G$ , and perform SGD on  $D$  for  $k$  iterations to increase  $f_{\text{acc}}$ .
  - Freeze  $D$ , and perform SGD on  $G$  for  $l$  iterations to decrease  $f_{\text{acc}}$ .

# Training GANs

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

- In practice, we train  $D$  and  $G$  *iteratively*:
  - Freeze  $G$ , and perform SGD on  $D$  for  $k$  iterations to increase  $f_{\text{acc}}$ .
  - Freeze  $D$ , and perform SGD on  $G$  for  $l$  iterations to decrease  $f_{\text{acc}}$ .

**Improve  $G$  for a fixed forgery detector  $D$ .**

# Difficulty in training

- GANs are renowned for being difficult to train:
  1. How to choose  $k$ ,  $l$ ? More hyperparameters to optimize.

# Difficulty in training

- GANs are renowned for being difficult to train:
  1. How to choose  $k$ ,  $l$ ? More hyperparameters to optimize.
  2. We will probably never reach the equilibrium where  $G$  exactly produces  $P_{\text{data}}(\mathbf{x})$  and  $D$ 's accuracy is 0.5.
    - What kind of “training curve” for  $D$ ,  $G$  should we expect?
    - If  $D$  gets too good too fast, then  $G$  may never have a chance to improve.

# Difficulty in training

- GANs are renowned for being difficult to train:

**3. Mode collapse** —  $G$  generates realistic data but only for a *subset* of the domain of  $P_{\text{data}}(\mathbf{x})$ , e.g.:



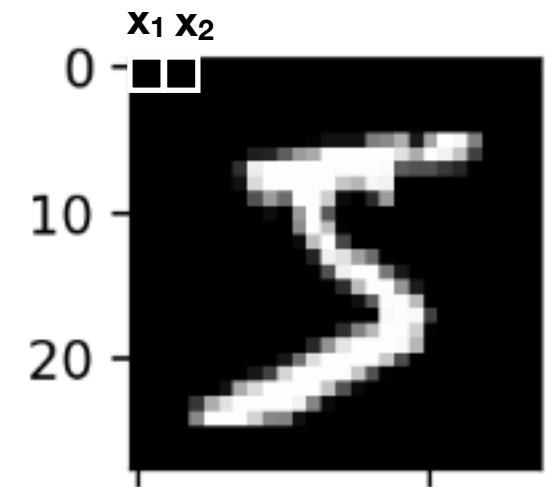
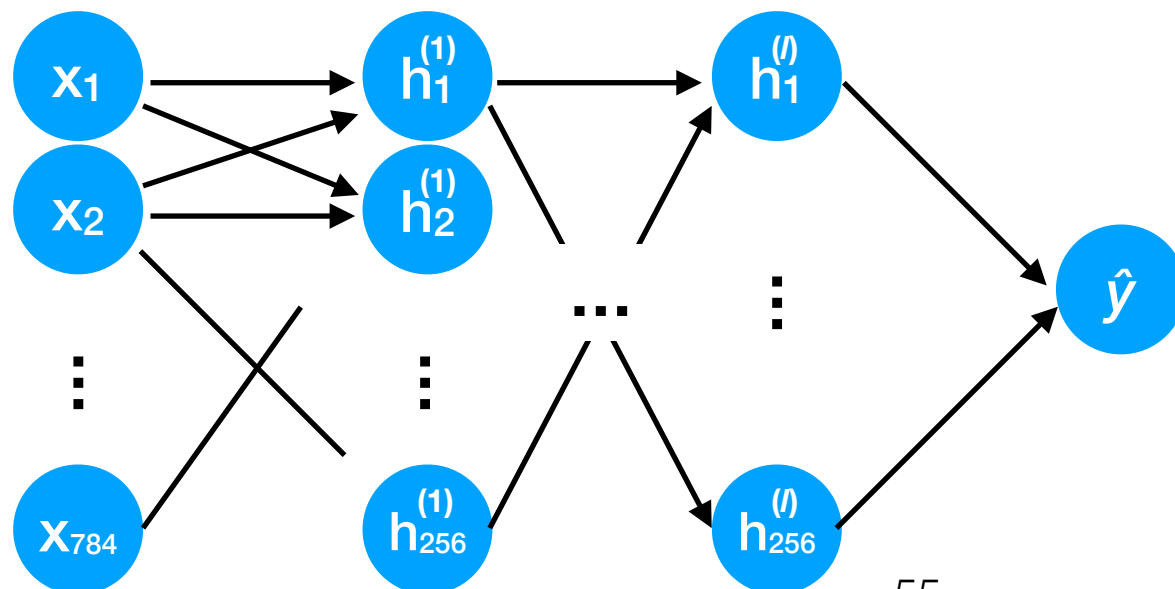
# Difficulty in training

- GANs are renowned for being difficult to train:

**4. Neuron co-adaptation** — training gets stuck because multiple NN pathways rely on each other too much.

- Consider an MNIST image near the borders: what property do pixels  $x_1, x_2$  have?

- $x_1 = x_2 = 0$  ?



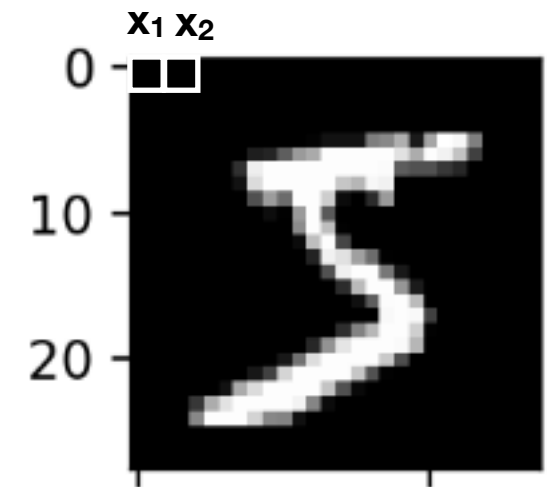
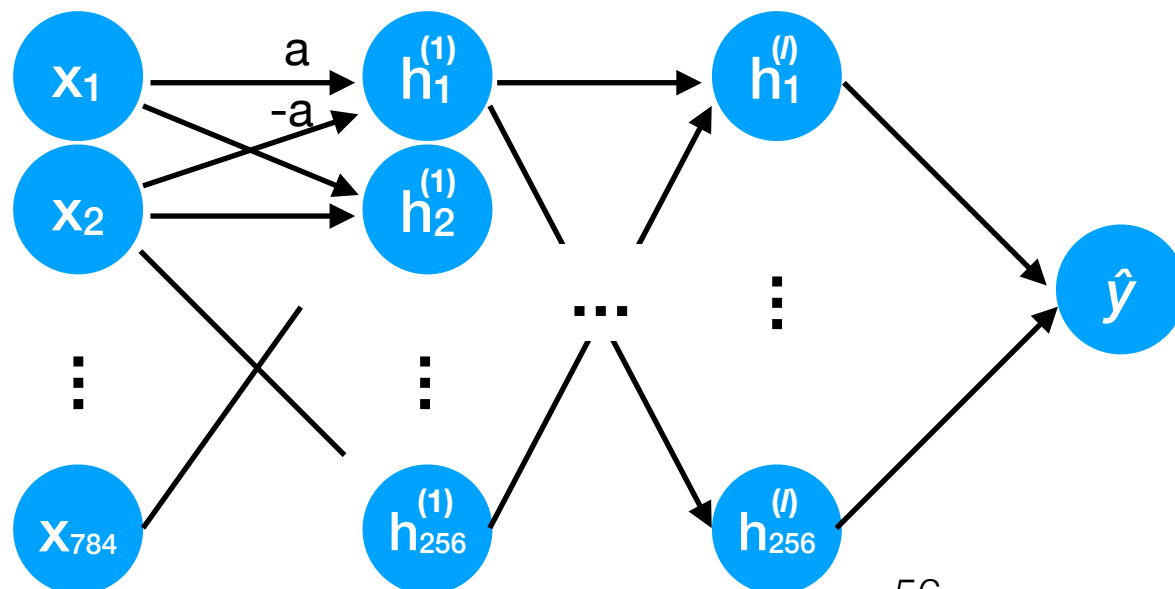
# Difficulty in training

- GANs are renowned for being difficult to train:

**4. Neuron co-adaptation** — training gets stuck because multiple NN pathways rely on each other too much.

- Consider an MNIST image near the borders: what property do pixels  $x_1, x_2$  have?

- $ax_1 - ax_2 = 0$  ?

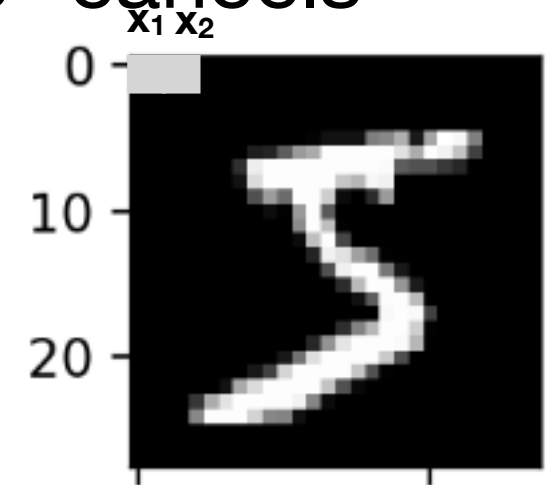
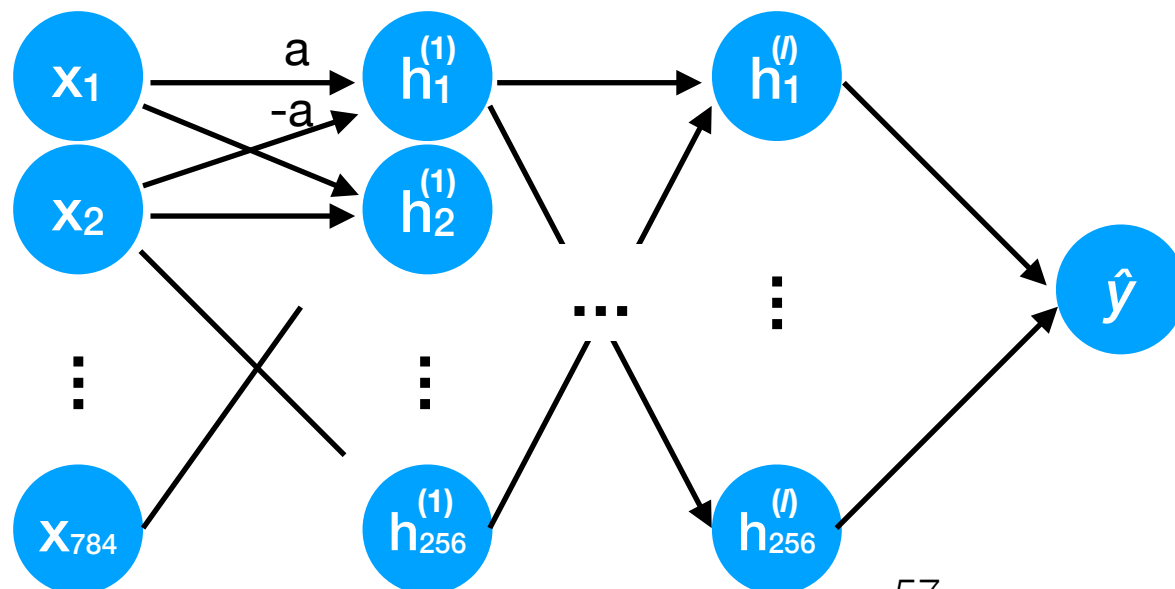


# Difficulty in training

- GANs are renowned for being difficult to train:

**4. Neuron co-adaptation** — training gets stuck because multiple NN pathways rely on each other too much.

- In the latter case,  $D$  gives feedback to  $G$  that images are “ok” as long as the background noise “cancels” itself, e.g.:

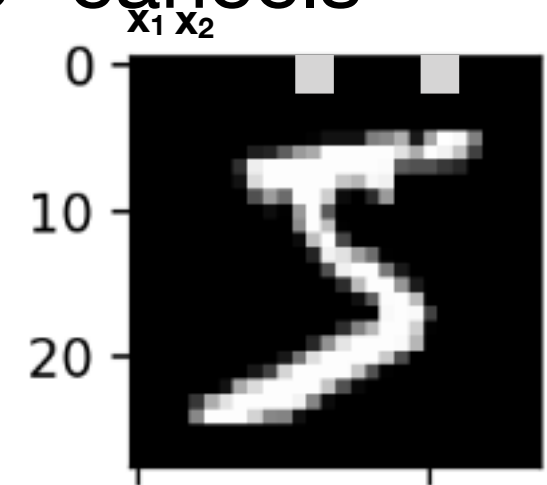
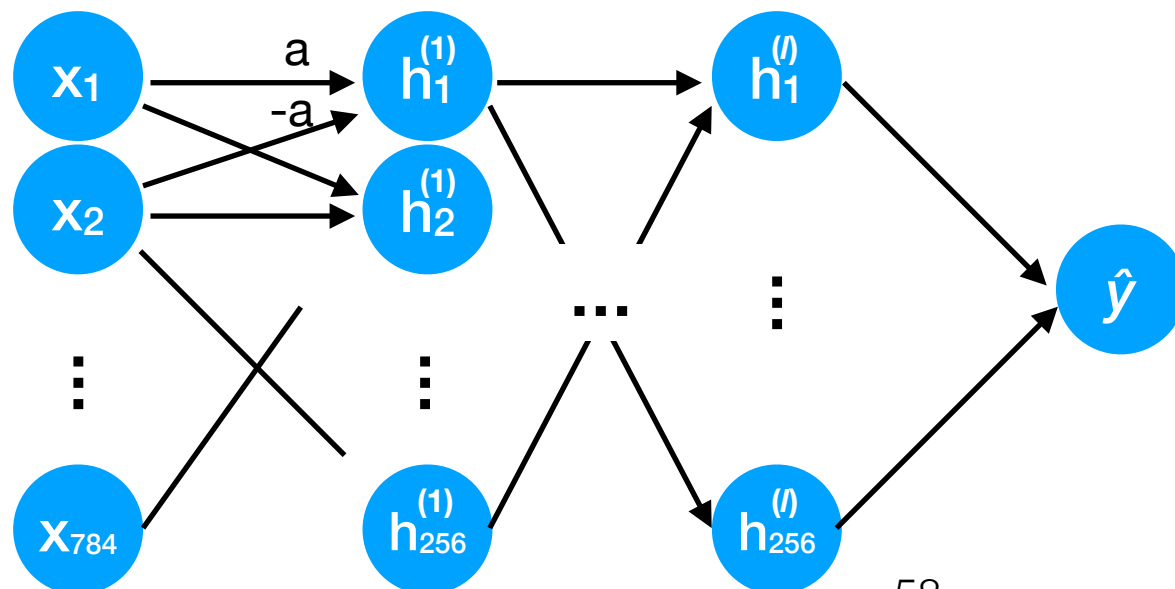


# Difficulty in training

- GANs are renowned for being difficult to train:

**4. Neuron co-adaptation** — training gets stuck because multiple NN pathways rely on each other too much.

- In the latter case,  $D$  gives feedback to  $G$  that images are “ok” as long as the background noise “cancels” itself, e.g.:

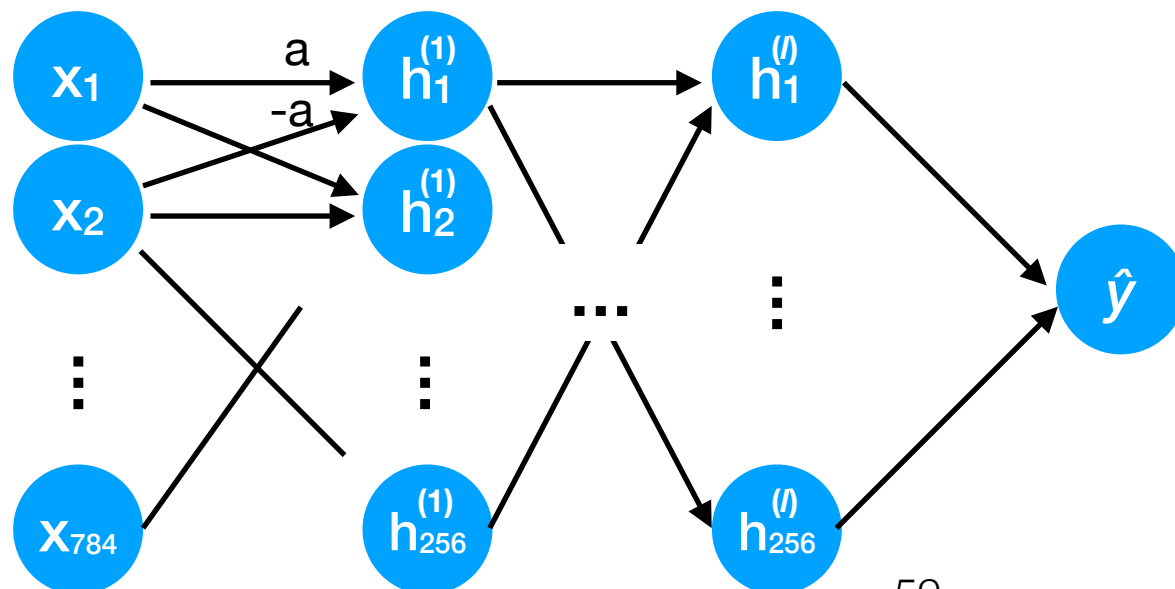


# Difficulty in training

- GANs are renowned for being difficult to train:

**4. Neuron co-adaptation** — training gets stuck because multiple NN pathways rely on each other too much.

- In the latter case,  $D$  gives feedback to  $G$  that images are “ok” as long as the background noise “cancels” itself, e.g.:

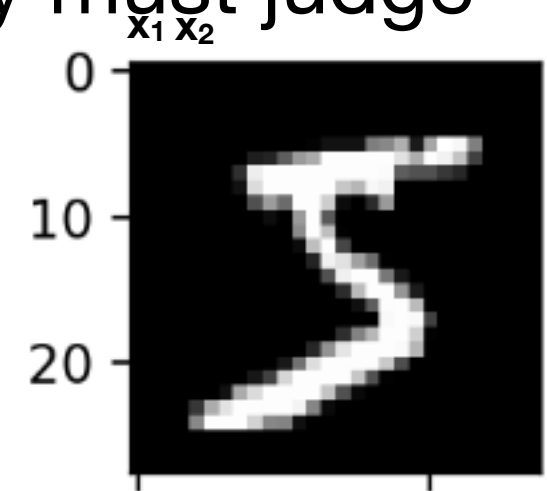
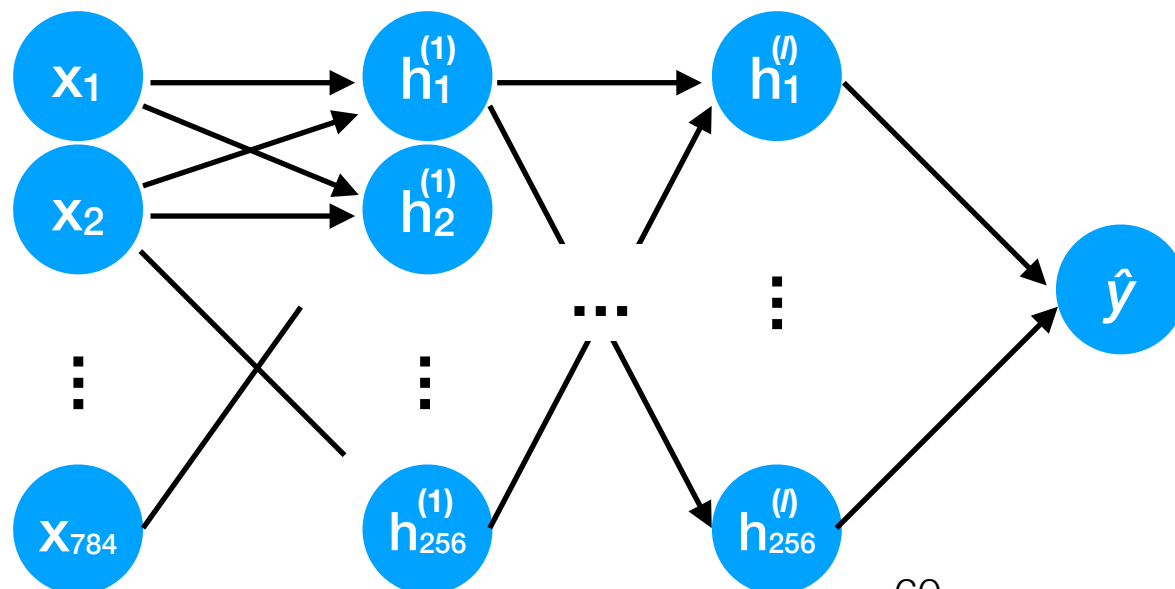


# Difficulty in training

- GANs are renowned for being difficult to train:

**4. Neuron co-adaptation** — training gets stuck because multiple NN pathways rely on each other too much.

- To prevent this from occurring, we can use dropout on the input layer  $\mathbf{x}$ , so that each pathway must judge independently if the image is a fake.



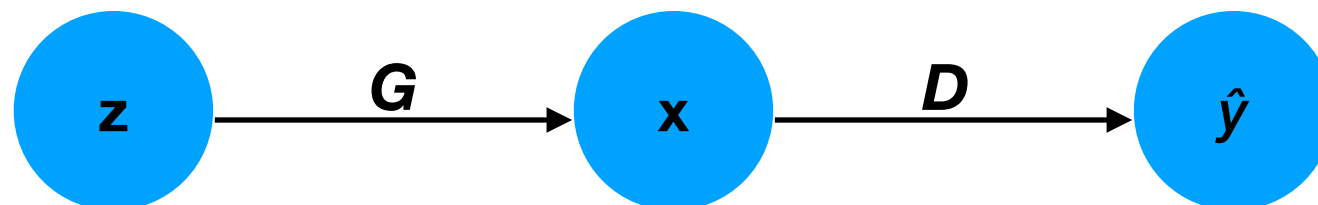
# Closer look at $f_{\text{acc}}$

- Consider the loss term for fake data:

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

- What happens early during training, when  $G$  is not very good (but  $D$  typically is fairly good)?

$$\nabla_{\theta_G} \log(1 - D(\mathbf{x})) = -\frac{1}{1 - D(\mathbf{x})} \frac{\partial D}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \theta_G}$$



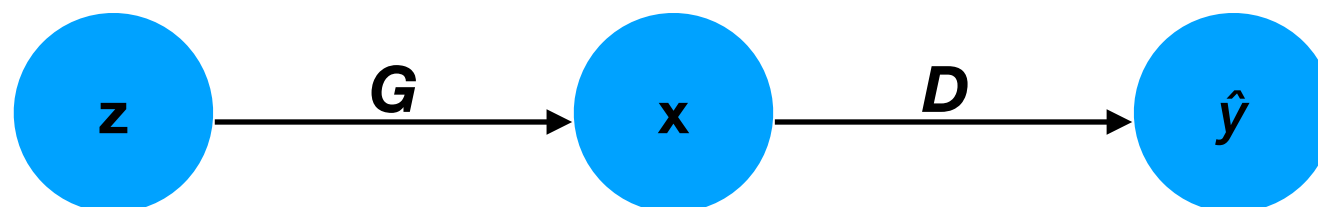
# Closer look at $f_{\text{acc}}$

- Consider the loss term for fake data:

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

- What happens early during training, when  $G$  is not very good (but  $D$  typically is fairly good)?

- $D \approx 0$  and  $dD/d\mathbf{x} \approx \mathbf{0}$ . 
$$\nabla_{\theta_G} \log(1 - D(\mathbf{x})) = -\frac{1}{1 - D(\mathbf{x})} \frac{\partial D}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \theta_G}$$
$$\approx -1 \frac{\partial D}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \theta_G}$$



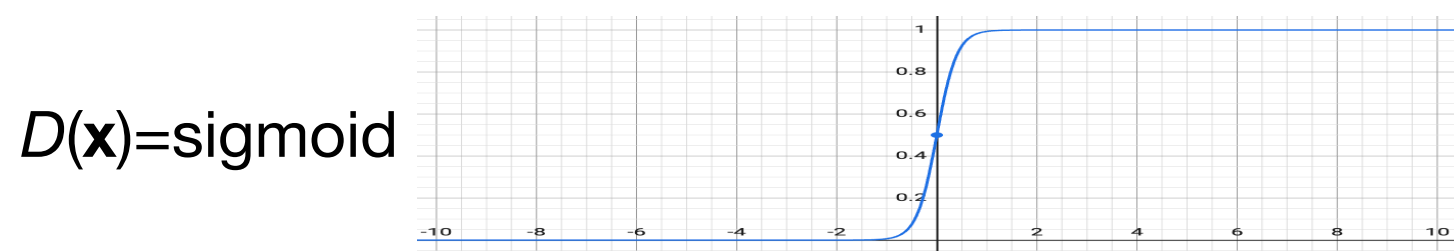
# Closer look at $f_{\text{acc}}$

- Consider the loss term for fake data:

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

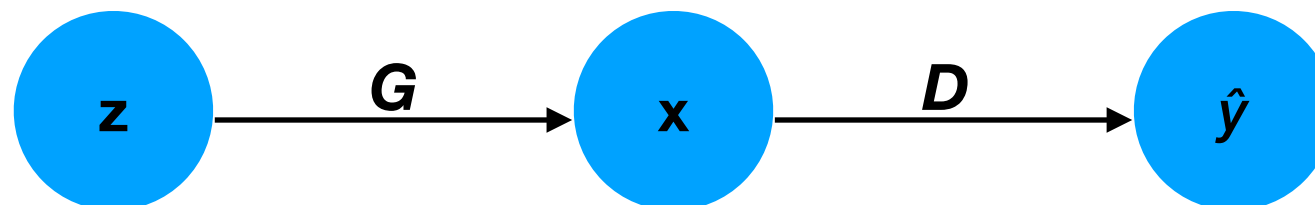
- What happens early during training, when  $G$  is not very good (but  $D$  typically is fairly good)?

- $D \approx 0$  and  $dD/d\mathbf{x} \approx \mathbf{0}$ .  $\nabla_{\theta_G} \log(1 - D(\mathbf{x})) = -\frac{1}{1 - D(\mathbf{x})} \frac{\partial D}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \theta_G}$



$$\approx -1 \frac{\partial D}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \theta_G}$$

$$\approx -1 \times \text{small} \times \frac{\partial \mathbf{x}}{\partial \theta_G}$$



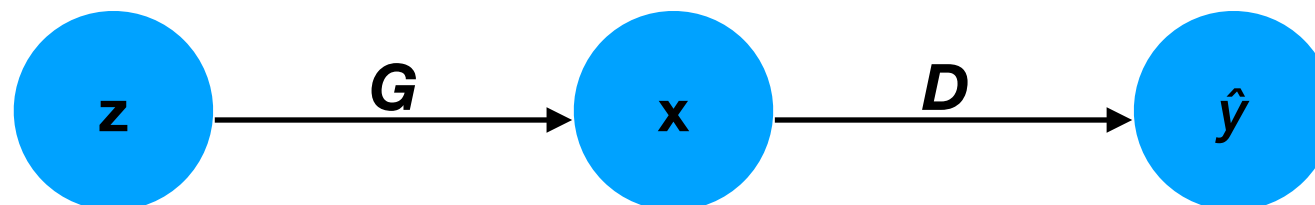
# Closer look at $f_{\text{acc}}$

- To accelerate training, we can use a different loss for the fake data that expresses the same goal but trains faster.

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [-\log(D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

New loss term

$$\begin{aligned} \nabla_{\theta_G} -\log D(\mathbf{x}) &= -\frac{1}{D(\mathbf{x})} \frac{\partial D}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \theta_G} \\ &\approx -\frac{1}{\text{small}} \times \text{small} \times \frac{\partial \mathbf{x}}{\partial \theta_G} \end{aligned}$$



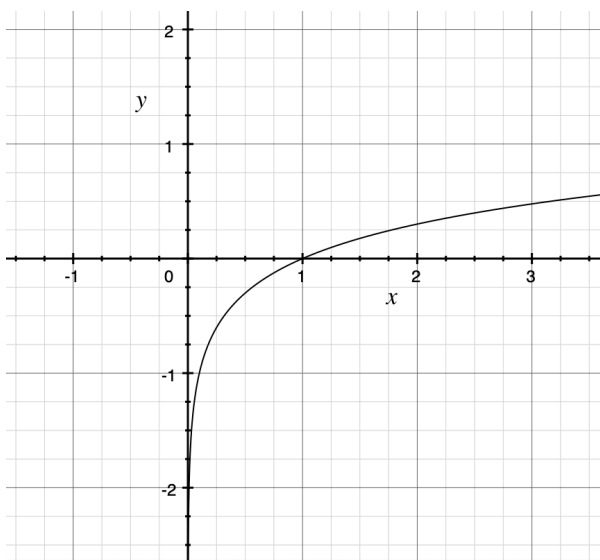
# Closer look at $f_{\text{acc}}$

- To accelerate training, we can use a different loss for the fake data that expresses the same goal but trains faster.

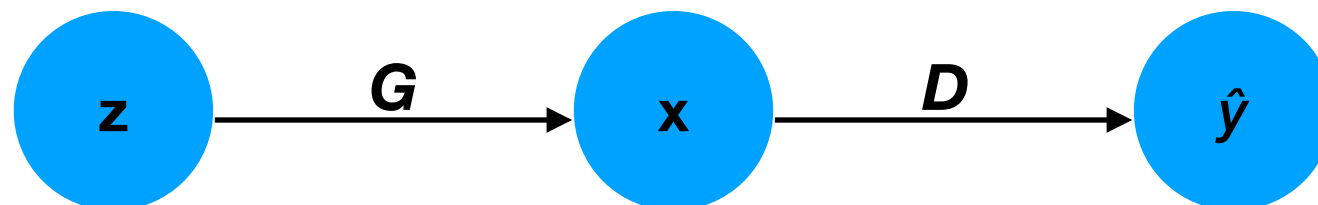
$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})} [-\log(D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

New loss term

- The reason is that the gradient of  $-\log(q)$  for  $q \approx 0$  is very large, whereas the gradient of  $\log(1-q) \approx 1$ .



$$\begin{aligned} \nabla_{\theta_G} -\log D(\mathbf{x}) &= -\frac{1}{D(\mathbf{x})} \frac{\partial D}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \theta_G} \\ &\approx -\frac{1}{\text{small}} \times \text{small} \times \frac{\partial \mathbf{x}}{\partial \theta_G} \end{aligned}$$



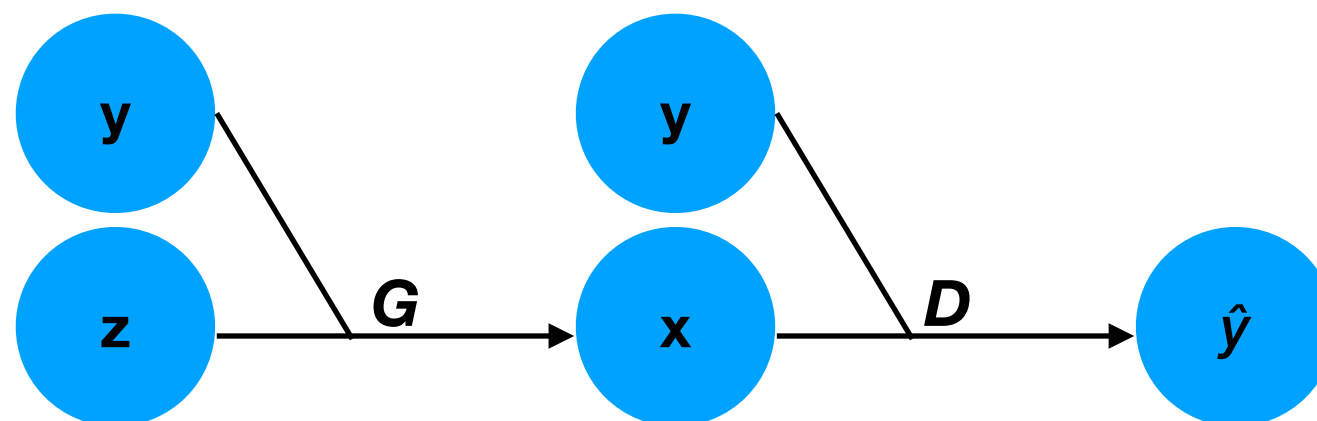
# Conditional data generation

# Generative AI is probabilistic

- For generative AI queries, there is usually more than just “one possible answer”, e.g.:
  - “Compose me a new song.”  $\mathbf{x} \sim p(\mathbf{x})$
  - “Create a photo of bears hugging bunnies.”  $\mathbf{x} \sim p(\mathbf{x} \mid \mathbf{y})$
  - “Complete the sentence: Abraham Lincoln was born \_\_\_\_\_.”  $\arg \max_{\mathbf{x}} p(\mathbf{x} \mid \mathbf{y})$
- We typically want to either **sample** a plausible example from the distribution, or find the **most likely example** from the entire distribution.

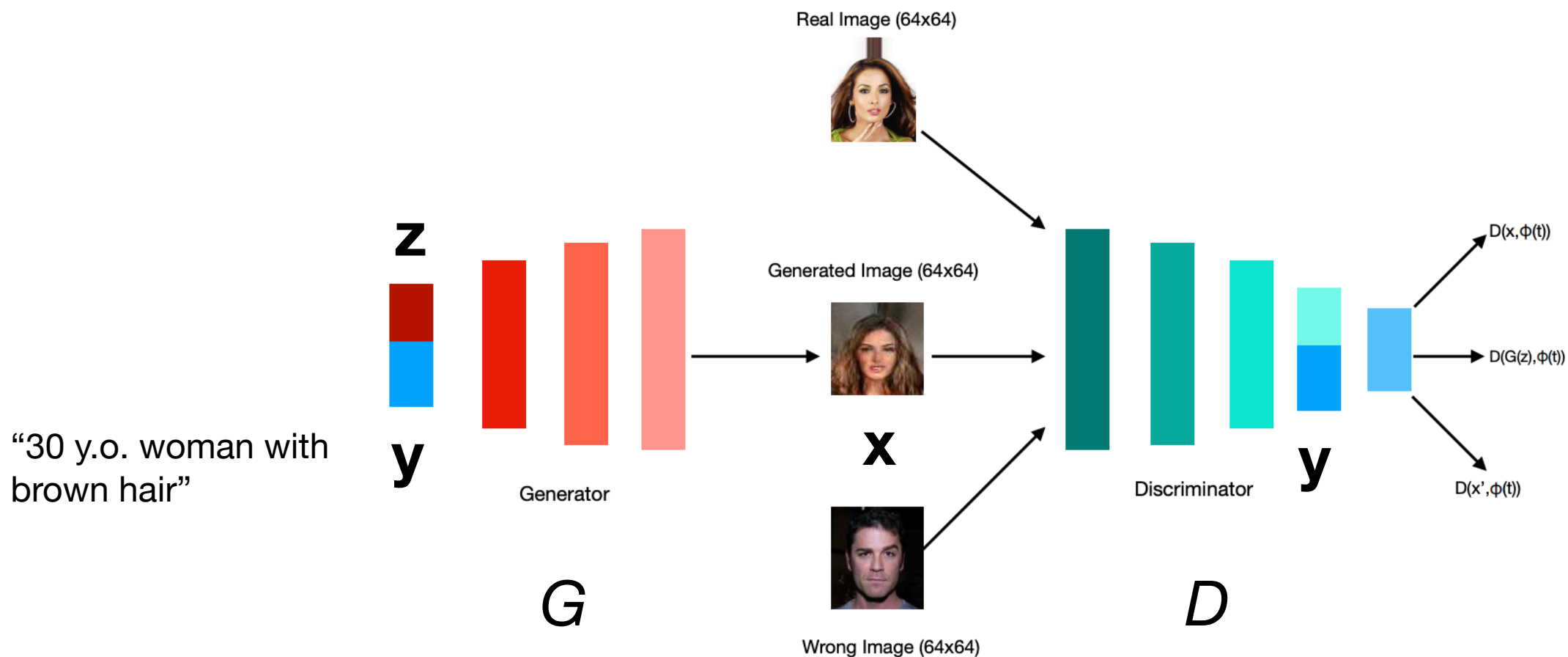
# Conditional GANs

- One GAN variant is a **conditional GAN (c-GAN)**:
  - $G$  also accepts a feature vector  $\mathbf{y}$  (e.g., 1-hot encoding of MNIST class) that specifies what *kind* of data to generate.
  - $D$  also accepts  $\mathbf{y}$  to help discriminate a particular kind of real from fake data.



# Applications

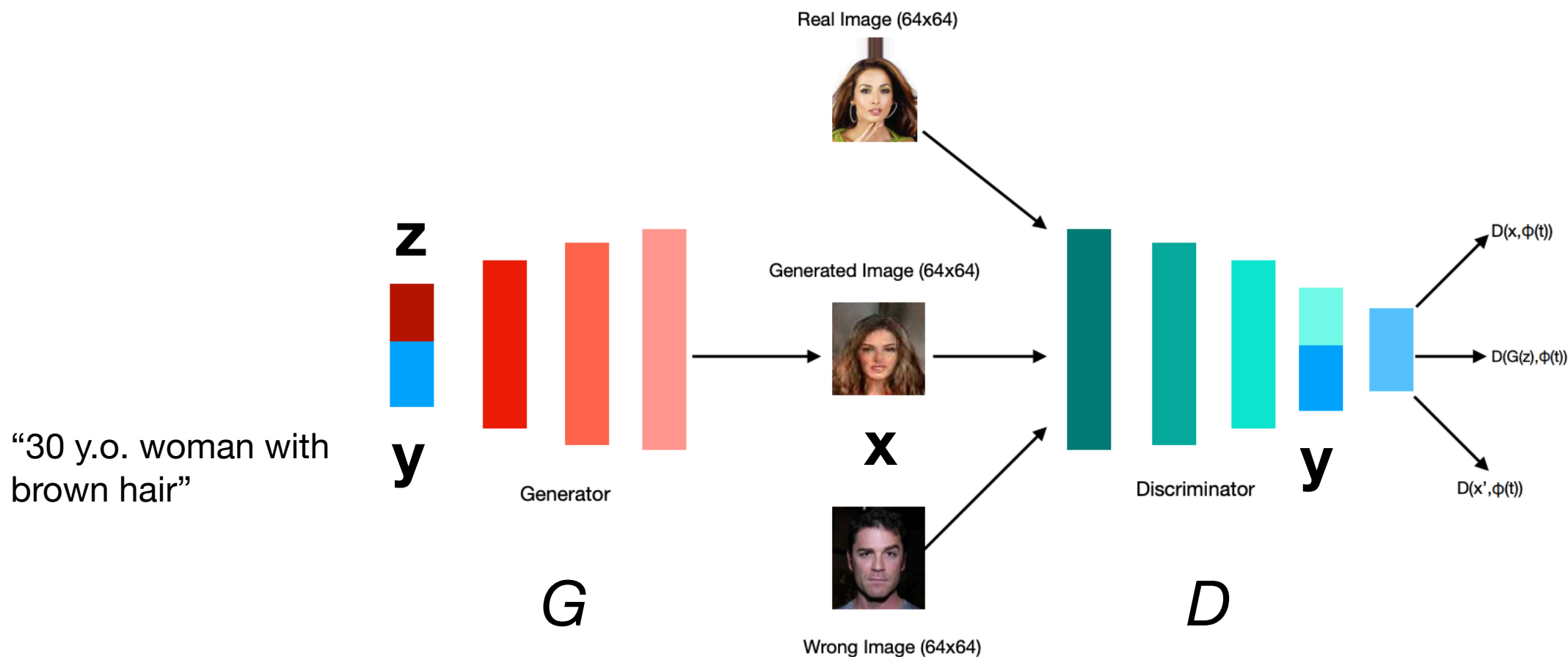
- Face generation from a text description (Nasir et al. 2019):



- Input to  $G$  is a sentence embedding  $y$  describing the desired image and a random noise vector  $z$ .
- Input to  $D$  is a face image  $x$  and sentence embedding  $y$ .

# Applications

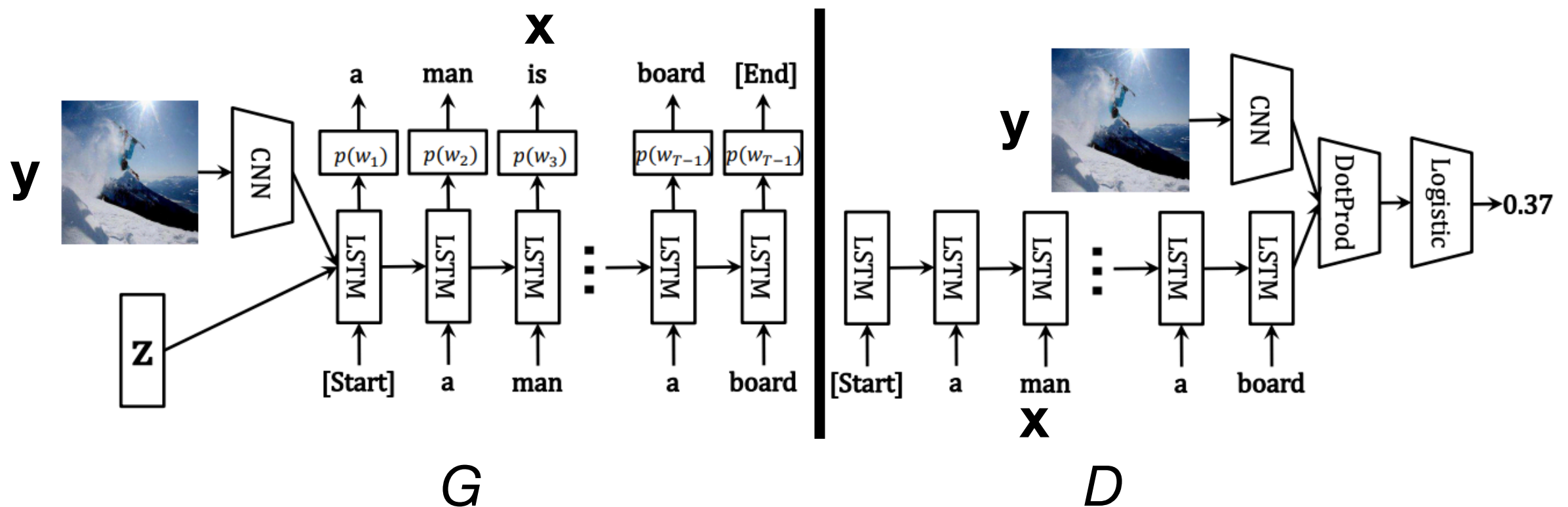
- Face generation from a text description (Nasir et al. 2019):



- GAN is trained on (caption, image) data from CelebA dataset.
- “Fake” example could be a generated image, or a mismatch between an image and its caption.

# Applications

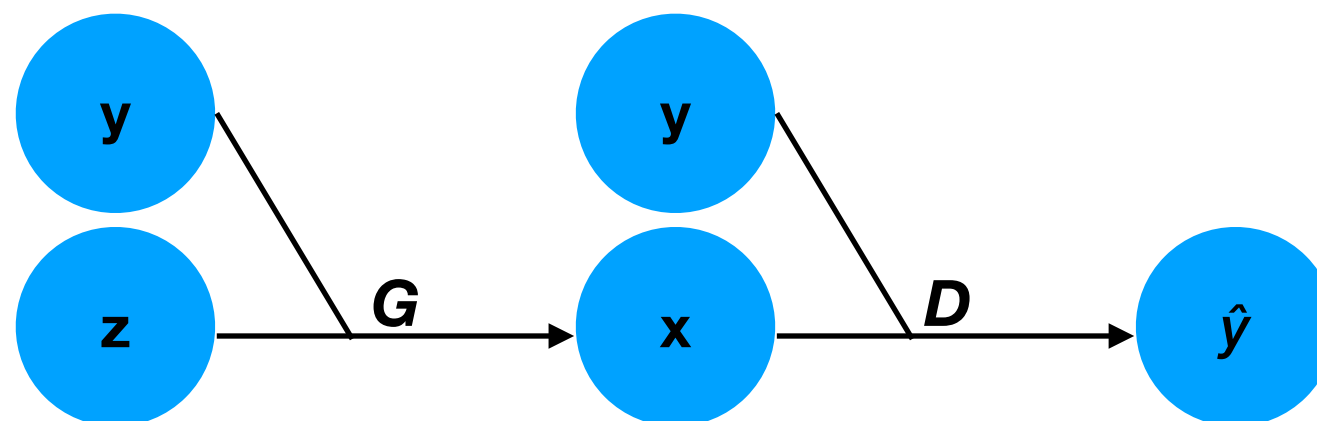
- Image captioning (Dai et al. 2017):



- Input to  $G$  is an image  $y$  and a random noise vector  $z$ .
- Input to  $D$  is an image  $y$  and a caption  $x$ .

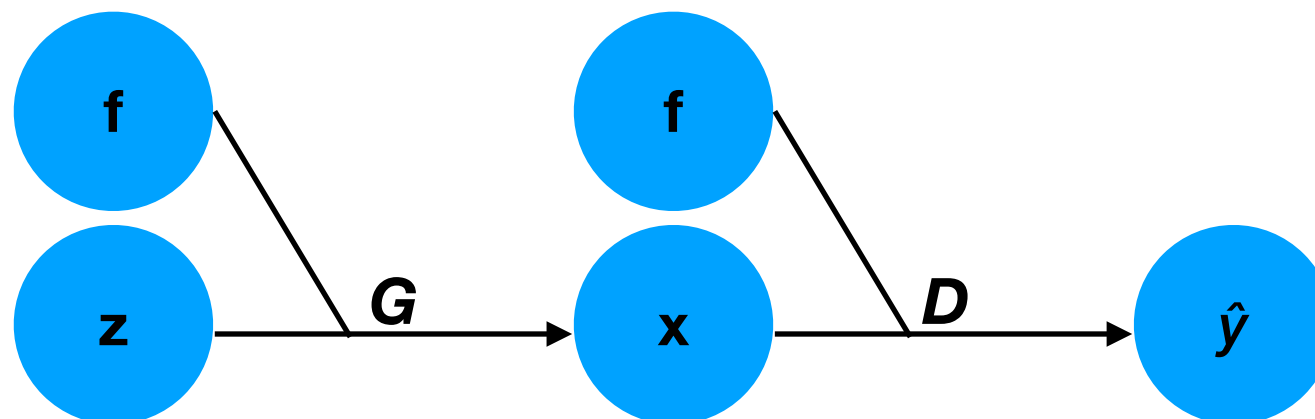
# Conditional GANs

- Why is  $\mathbf{y}$  necessary in  $G$ ?
- Why is  $\mathbf{y}$  necessary in  $D$ ?
- Why better than just training a separate GAN for each  $\mathbf{y}$ ?



# Conditional GANs

- Why is  $\mathbf{y}$  necessary in  $G$ ?
  - Need to know what class to generate.
- Why is  $\mathbf{y}$  necessary in  $D$ ?
  - Need to know what class was generated — a realistic 3 is still a fake 7!
- Why better than just training a separate GAN for each  $\mathbf{y}$ ?
  - Share knowledge across many classes.



# Accuracy metrics in image generation

# Accuracy metrics in image generation

- When training generative models (VAEs, GANs, etc.), it is useful to have an objective method to assess the quality of generated images.
- What do these models optimize?
  - VAE:
  - GAN:

# Accuracy metrics in image generation

- When training generative models (VAEs, GANs, etc.), it is useful to have an objective method to assess the quality of generated images.
- What do these models optimize?

- VAE: minimize KL divergence and  $L_2$  reconstruction loss

$$-D_{\text{KL}}(Q_{\phi}(\mathbf{z} \mid \mathbf{x}) \parallel P(\mathbf{z})) + \mathbb{E}_{Q_{\phi}}[\log P_{\theta}(\mathbf{x} \mid \mathbf{z})]$$

- GAN: minimax of  $D$ 's accuracy in distinguishing fake/real

$$f_{\text{acc}}(\theta_G, \theta_D) = \mathbb{E}_{\mathbf{x} \sim P_{\text{data}}(\mathbf{x})}[\log D_{\theta_D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim P(\mathbf{z})}[\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

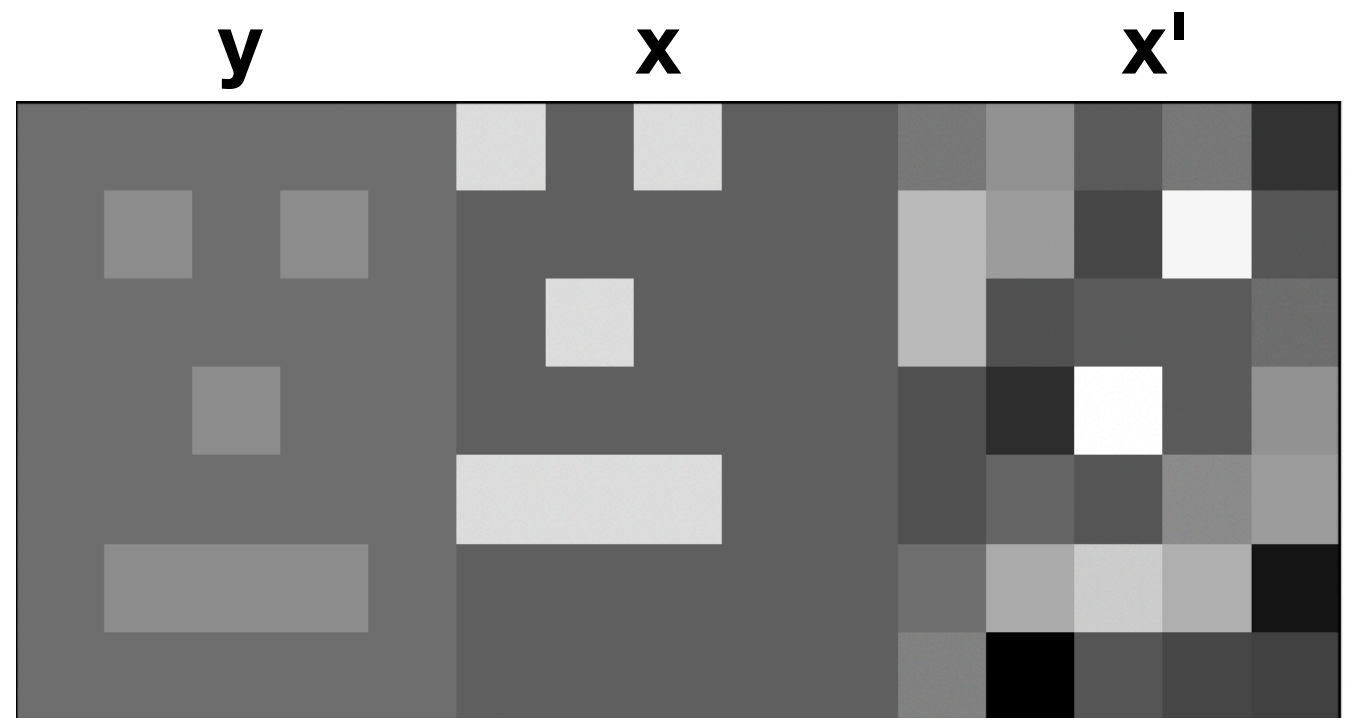
# Accuracy metrics in image generation

- For judging image quality, these metrics are useful but have their own particular issues:
- GAN discrimination accuracy of fake/real is affected by mode collapse.
- $L_2$  reconstruction loss leads to blurry images and also fails to capture how humans perceive image differences.



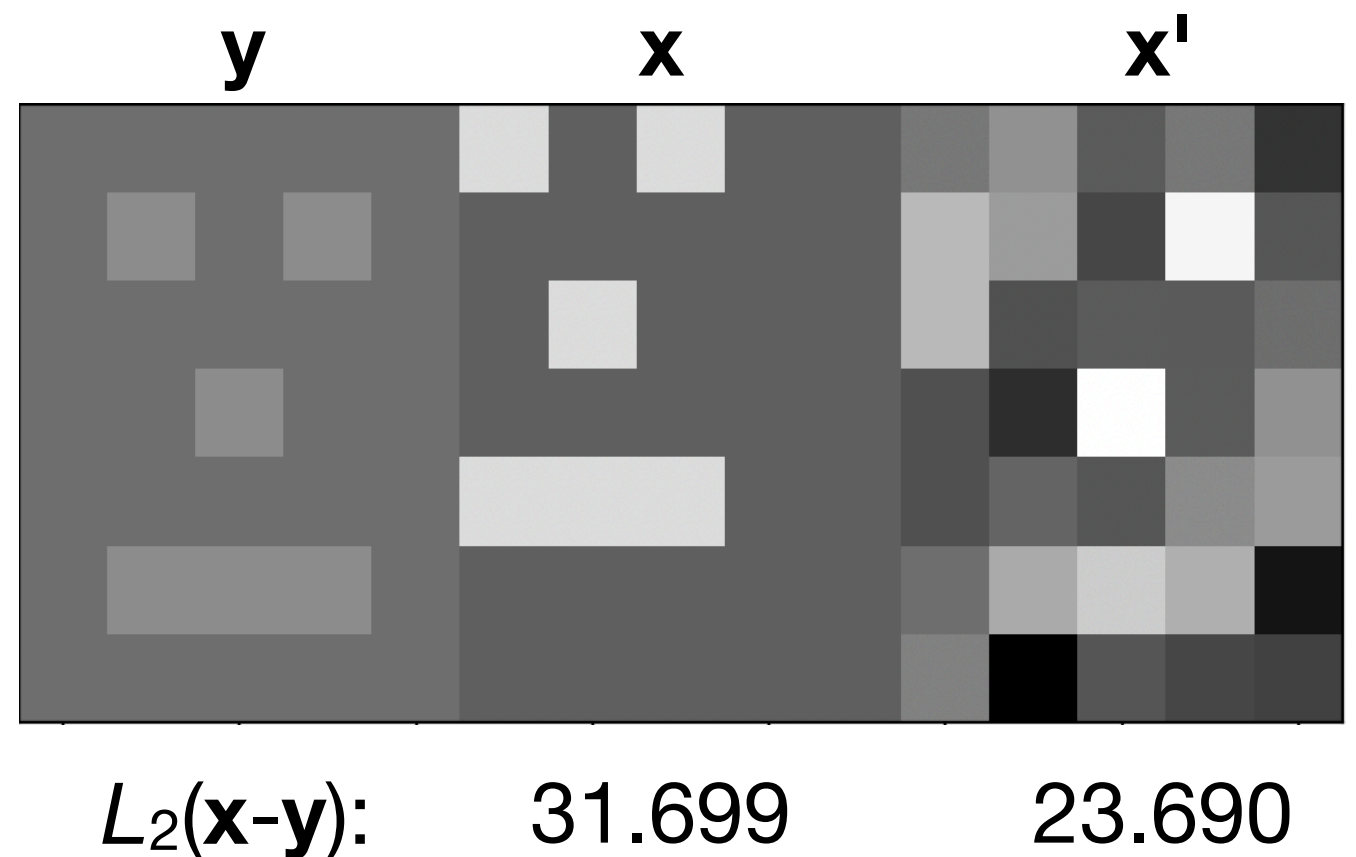
# $L_2$ reconstruction loss

- Suppose you wanted to generate an image  $\mathbf{x}$  that looks similar to the specified  $\mathbf{y}$ .
- Which one looks more similar to  $\mathbf{y}$ :  $\mathbf{x}$  or  $\mathbf{x}'$ ?



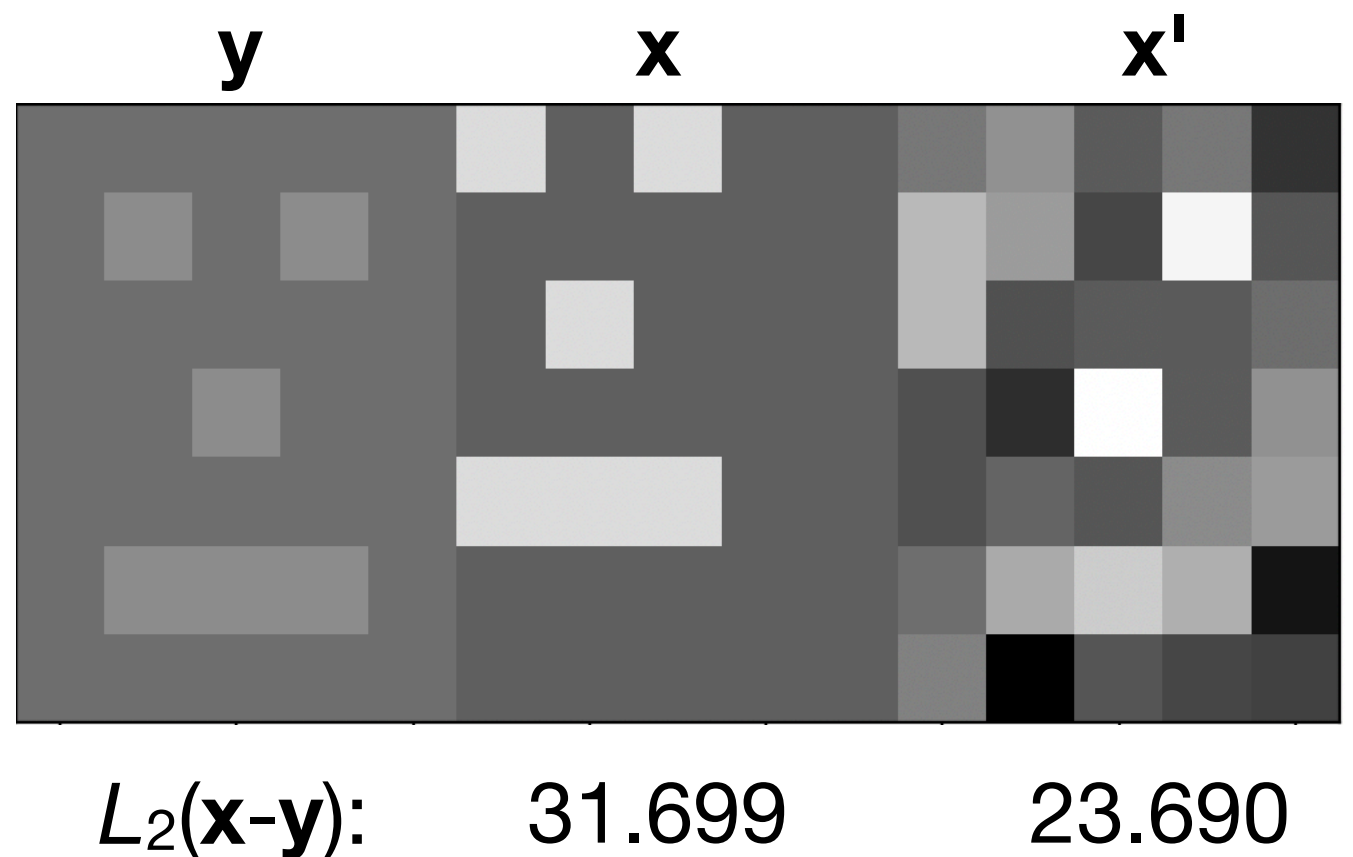
# $L_2$ reconstruction loss

- Suppose you wanted to generate an image  $\mathbf{x}$  that looks similar to the specified  $\mathbf{y}$ .
- Which one looks more similar to  $\mathbf{y}$ :  $\mathbf{x}$  or  $\mathbf{x}'$ ?
- What does the  $L_2$  distance say?



# $L_2$ reconstruction loss

- Suppose you wanted to generate an image  $\mathbf{x}$  that looks similar to the specified  $\mathbf{y}$ .
- Which one looks more similar to  $\mathbf{y}$ :  $\mathbf{x}$  or  $\mathbf{x}'$ ?
- What does the  $L_2$  distance say?
- $\mathbf{x}$  is intuitively much better but has a worse  $L_2$  score than  $\mathbf{x}'$   
=> wrong metric!

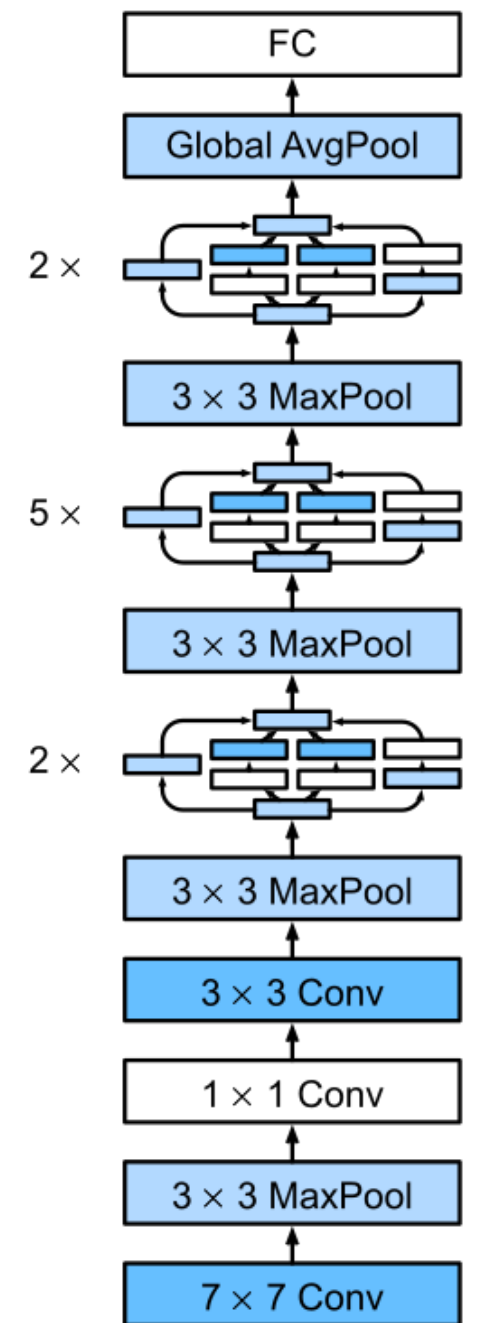


# Accuracy metrics in image generation

- What matters more in generated image quality is whether the objects are semantically meaningful and diverse.
- How can we assess the semantic content of an image?
  - High-quality CNNs to extract image content (e.g., for image recognition & object detection) have existed for 15+ years.

# Fréchet Inception Distance (FID)

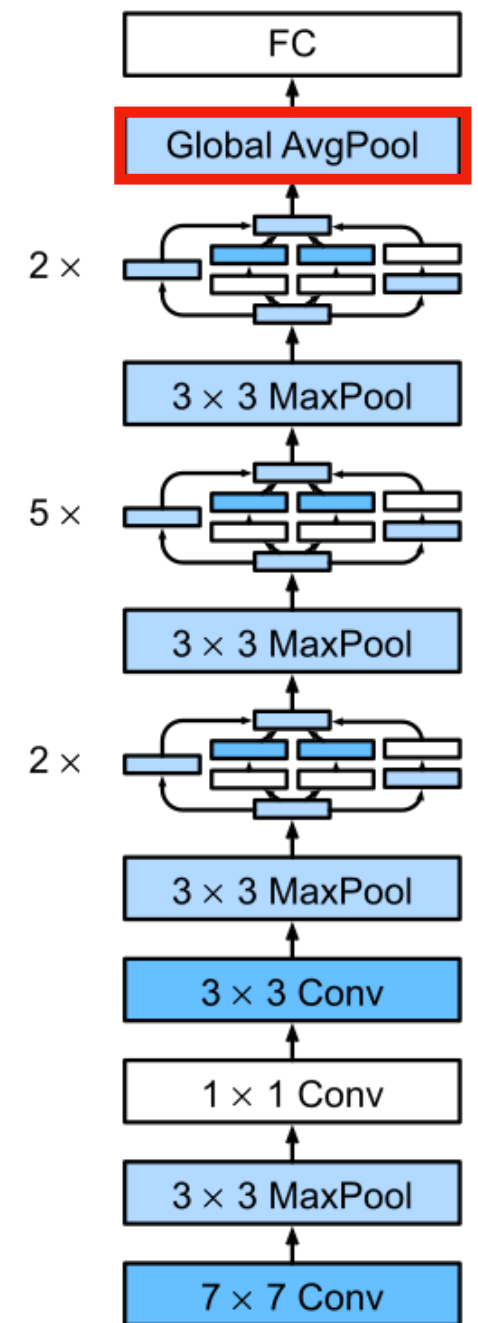
- With FID, the quality of a generated image dataset is measured by comparing *extracted image features* using a NN (Inception) to those of a *reference dataset* (e.g., ImageNet).
- Inception is a CNN (first built in 2014) with powerful feature extraction abilities.



[Inception CNN](#)

# Fréchet Inception Distance (FID)

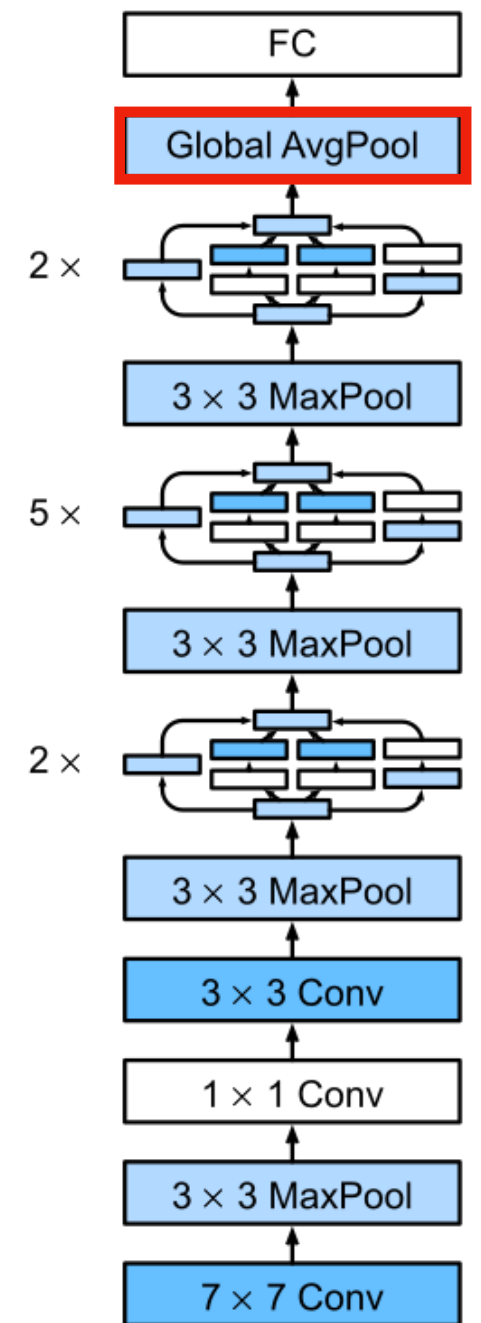
- Procedure:
  1. Extract feature vector  $\mathbf{f}^{(i)}$  from Inception's final pooling layer for each image  $i=1, \dots, n$ .



[Inception CNN](#)

# Fréchet Inception Distance (FID)

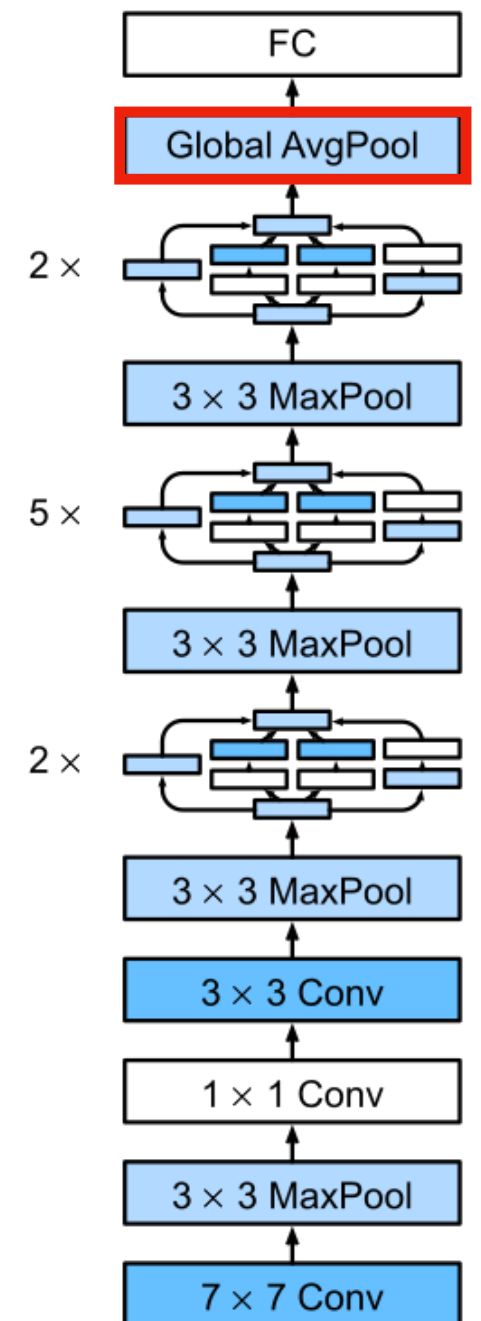
- Procedure:
  1. Extract feature vector  $\mathbf{f}^{(i)}$  from Inception's final pooling layer for each image  $i=1, \dots, n$ .
  2. Estimate a Gaussian distribution  $p(\mathbf{f})$  over  $\{\mathbf{f}^{(i)}\}_{i=1}^n$  with mean  $\mu_p$  and covariance  $\Sigma_p$ .



[Inception CNN](#)

# Fréchet Inception Distance (FID)

- Procedure:
  1. Extract feature vector  $\mathbf{f}^{(i)}$  from Inception's final pooling layer for each image  $i=1, \dots, n$ .
  2. Estimate a Gaussian distribution  $p(\mathbf{f})$  over  $\{\mathbf{f}^{(i)}\}_{i=1}^n$  with mean  $\mu_p$  and covariance  $\Sigma_p$ .
  3. Do the same for the reference dataset to obtain  $q(\mathbf{f})$  with mean  $\mu_q$  and covariance  $\Sigma_q$ .



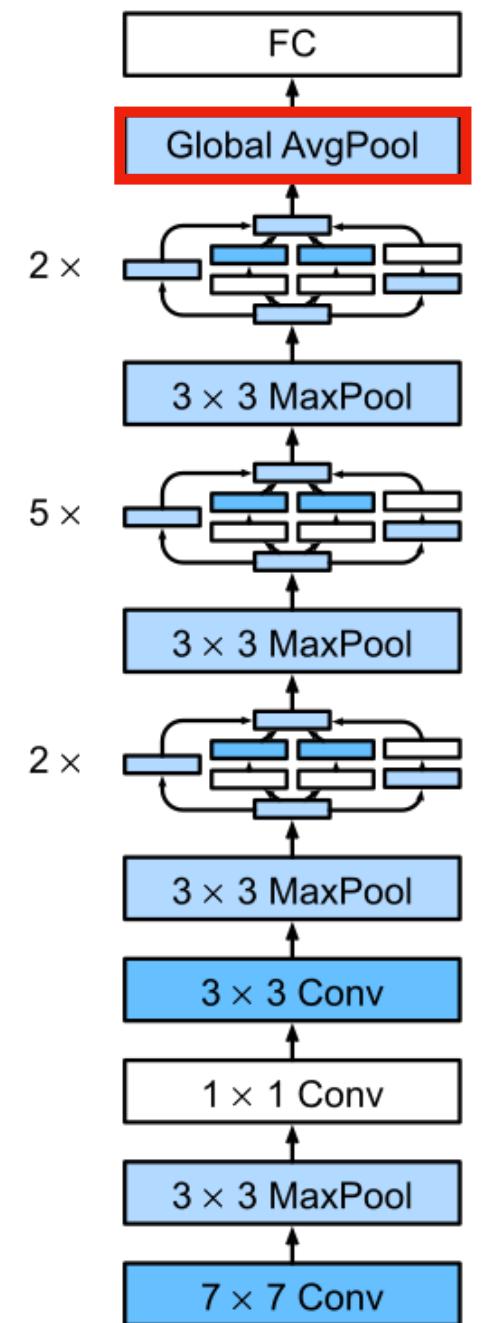
[Inception CNN](#)

# Fréchet Inception Distance (FID)

- Procedure:

1. Extract feature vector  $\mathbf{f}^{(i)}$  from Inception's final pooling layer for each image  $i=1, \dots, n$ .
2. Estimate a Gaussian distribution  $p(\mathbf{f})$  over  $\{\mathbf{f}^{(i)}\}_{i=1}^n$  with mean  $\mu_p$  and covariance  $\Sigma_p$ .
3. Do the same for the reference dataset to obtain  $q(\mathbf{f})$  with mean  $\mu_q$  and covariance  $\Sigma_q$ .
4. Compute the Fréchet distance  $d$  (another similarity metric between probability distributions) between  $p(\mathbf{f})$  and  $q(\mathbf{f})$ :

$$d(p, q) = \|\mu_p - \mu_q\|_2^2 + \text{tr} \left( \Sigma_p + \Sigma_q - 2(\Sigma_p \Sigma_q)^{\frac{1}{2}} \right)$$



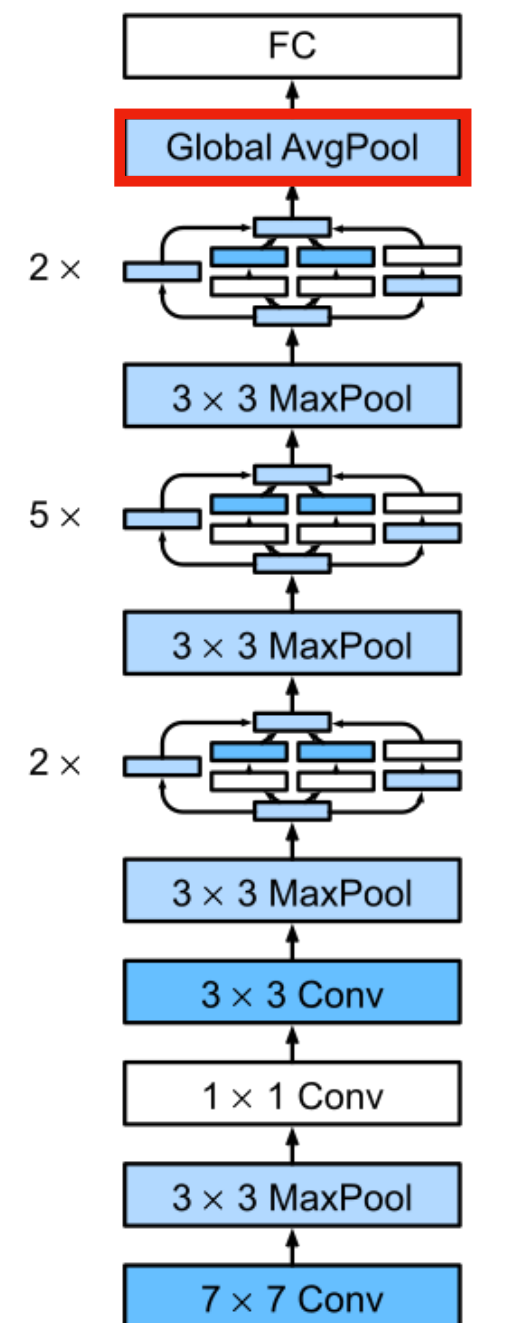
[Inception](#) CNN

# Fréchet Inception Distance (FID)

- Procedure:
  1. Extract feature vector  $\mathbf{f}^{(i)}$  from Inception's final pooling layer for each image  $i=1, \dots, n$ .
  2. Estimate a Gaussian distribution  $p(\mathbf{f})$  over  $\{\mathbf{f}^{(i)}\}_{i=1}^n$  with mean  $\mu_p$  and covariance  $\Sigma_p$ .
  3. Do the same for the reference dataset to obtain  $q(\mathbf{f})$  with mean  $\mu_q$  and covariance  $\Sigma_q$ .
  4. Compute the Fréchet distance  $d$  (another similarity metric between probability distributions) between  $p(\mathbf{f})$  and  $q(\mathbf{f})$ :

$$d(p, q) = \|\mu_p - \mu_q\|_2^2 + \text{tr} \left( \Sigma_p + \Sigma_q - 2(\Sigma_p \Sigma_q)^{\frac{1}{2}} \right)$$

[Show FID examples](#)

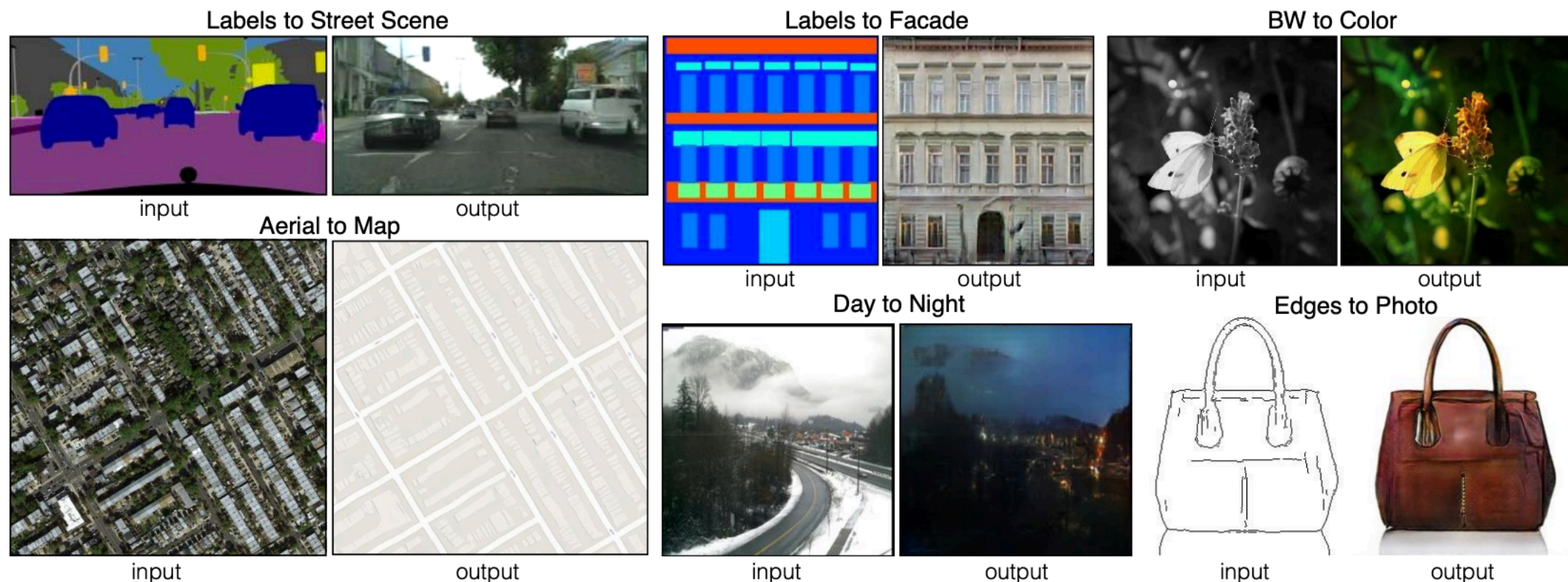


[Inception CNN](#)

**image->image  
translation**

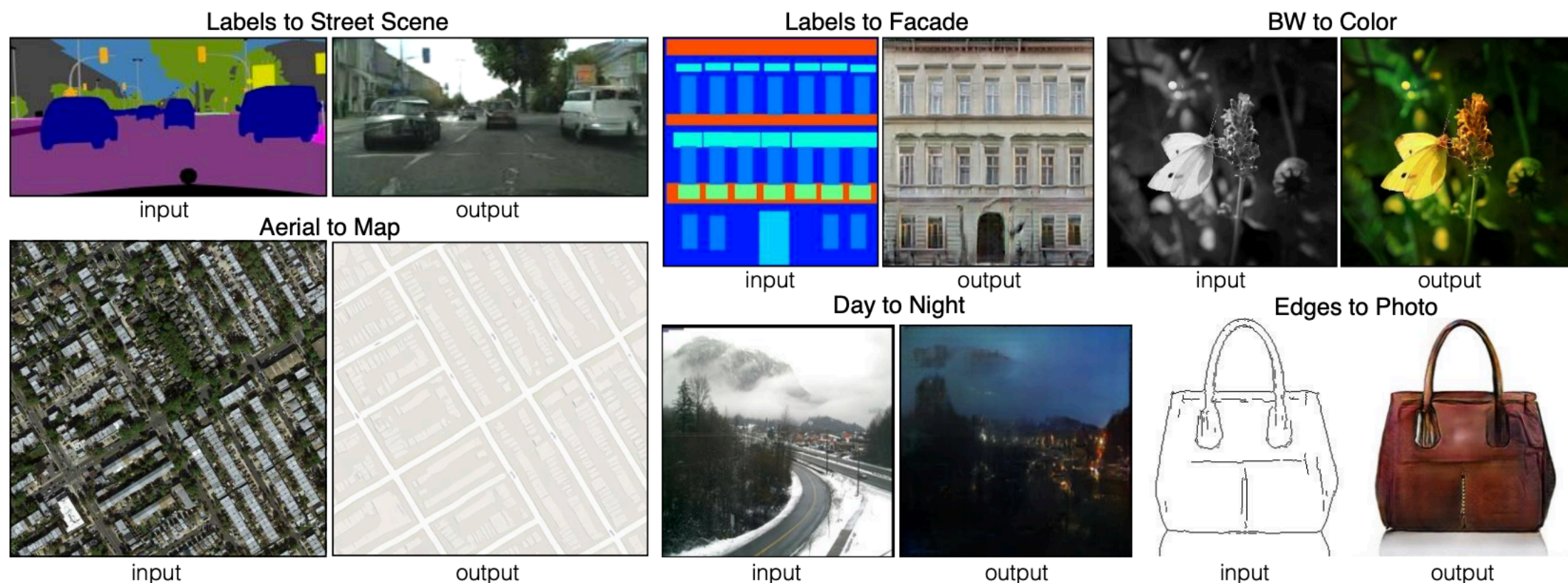
# pix2pix (Isola et al. 2018)

- Goal: map an image  $y$  into a “stylized” version  $x$  of same size.
- Approach: train a **U-Net** as a **c-GAN** to distinguish between real and fake  $(x,y)$  pairs.
- Even though this is a “predictive” task, we can still pose it as a conditional generative task.
- Example tasks:



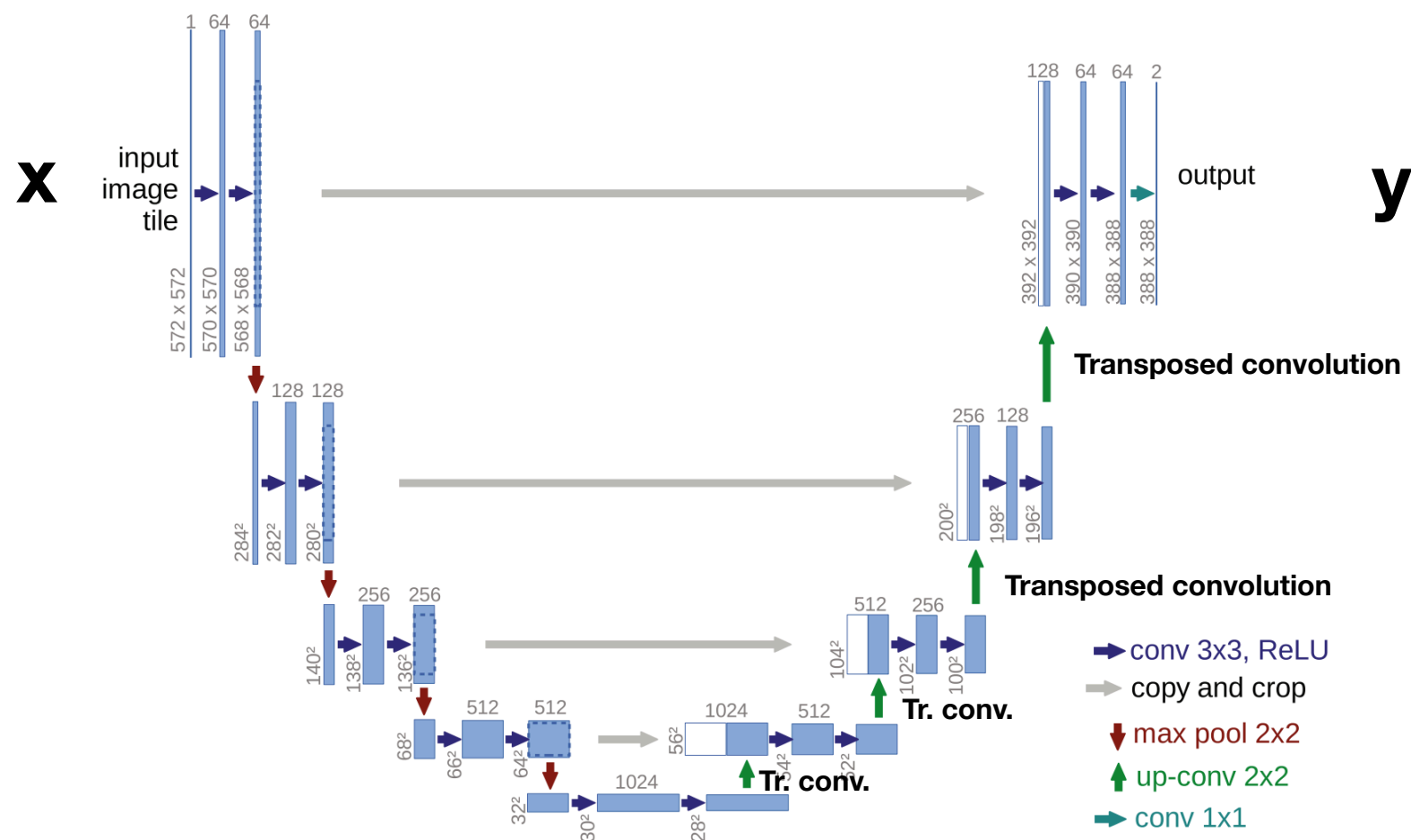
# pix2pix (Isola et al. 2018)

- Training a generative model to just minimize  $L_2$  loss will not work well.
- Instead, a c-GAN provides a custom loss function — learnt by the discriminator — that can give better quality.



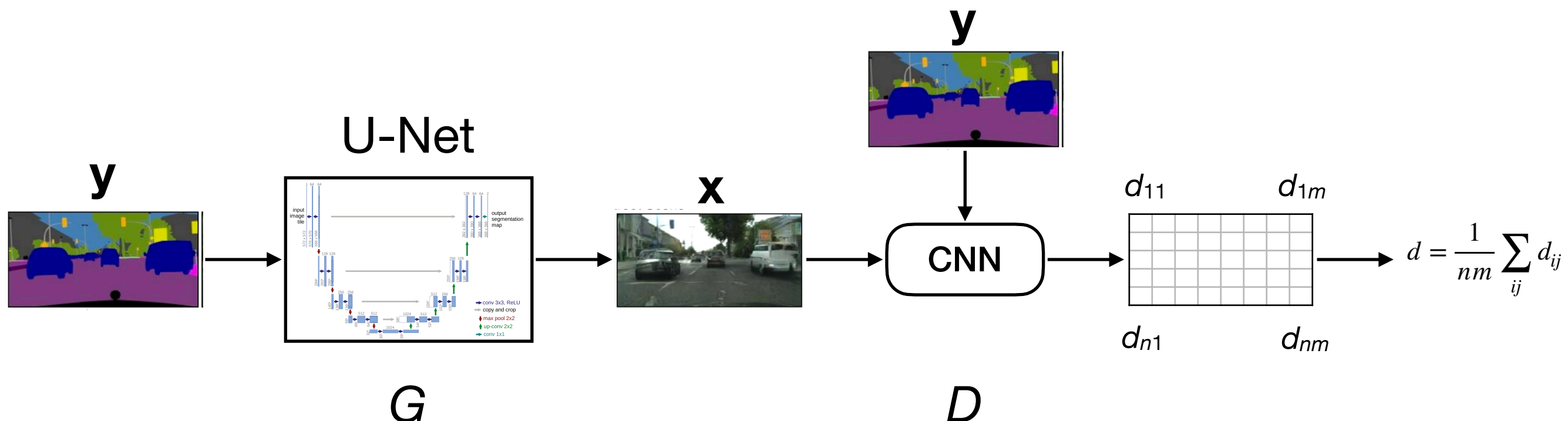
# U-Net

- A **U-Net** (Ronneberger et al. 2015) maps an image **x** into an image **y** of the same size.
- The architecture captures both *high-level semantics* via convolution & down-sampling, as well as *low-level details* with skip connections.



# pix2pix: architecture

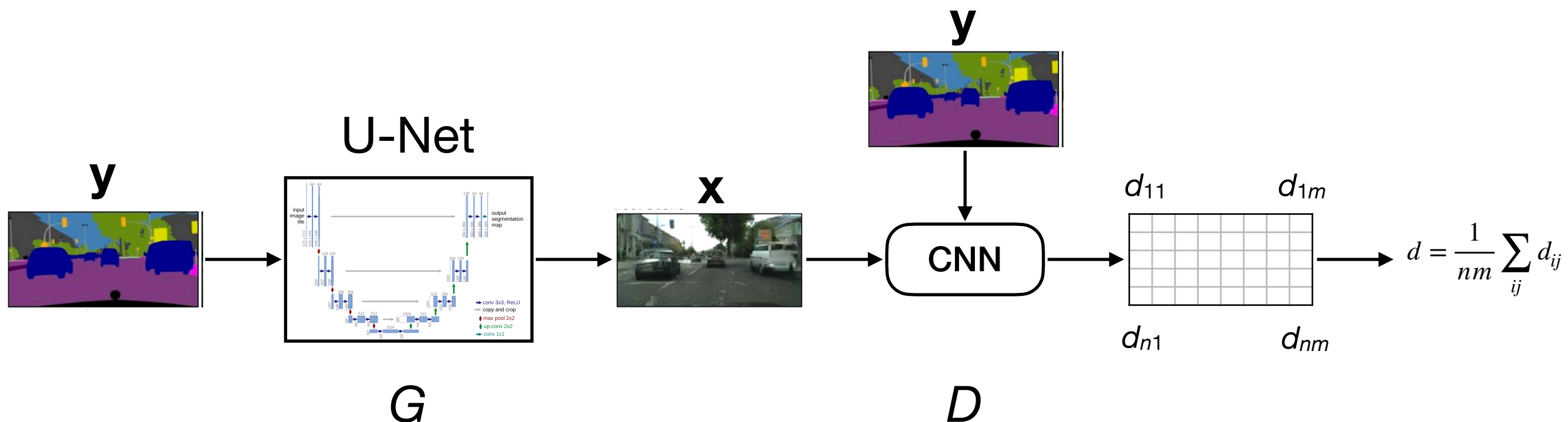
- $G$  is a U-Net and takes an input image  $y$  as context.
- In contrast to most c-GANs, pix2pix's  $G$  uses **no noise vector  $z$** , i.e., it is deterministic.
- $D$  takes image  $x$  (either generated or real) and context  $y$  to produce **patch-wise estimates** of fake/real, which are summed to produce a final discrimination score  $d$ .



# pix2pix: loss

- $G$  is trained both to maximally confuse  $D$  but also to minimize the  $L_1$  distance between  $\mathbf{x}$  and ground-truth image  $\mathbf{t}$ :

$$f(\mathbf{y}, \mathbf{t}; \theta_G) = f_{\text{cGAN}}(G(\mathbf{y}), \mathbf{y}, D) + \|G(\mathbf{y}) - \mathbf{t}\|_1$$

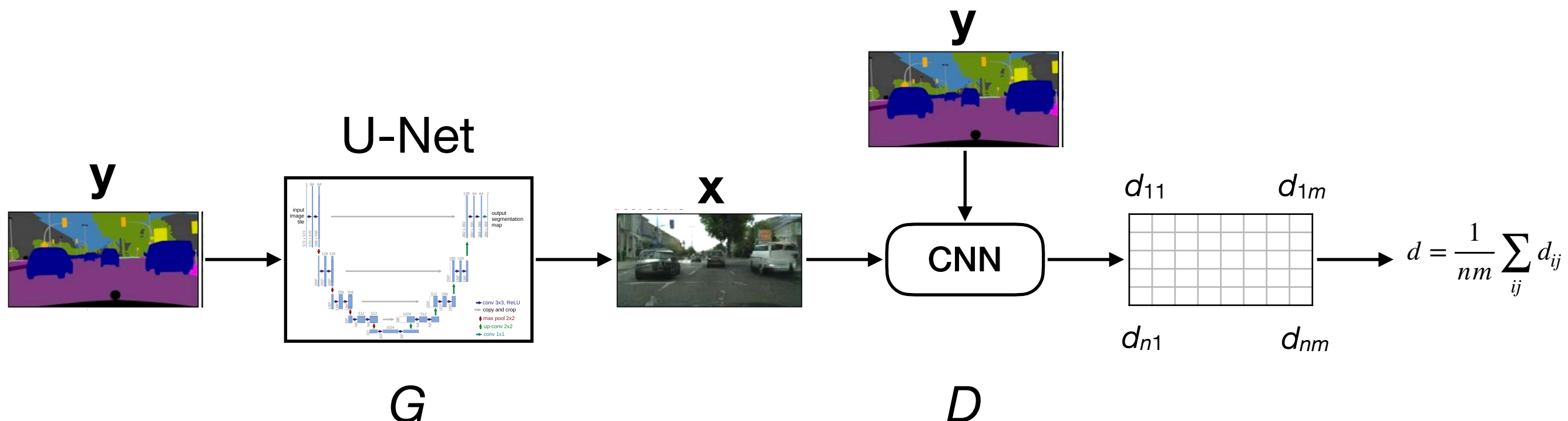


# pix2pix: loss

- $G$  is trained both to maximally confuse  $D$  but also to minimize the  $L_1$  distance between  $\mathbf{x}$  and ground-truth image  $\mathbf{t}$ :

$$f(\mathbf{y}, \mathbf{t}; \theta_G) = f_{\text{cGAN}}(G(\mathbf{y}), \mathbf{y}, D) + \|G(\mathbf{y}) - \mathbf{t}\|_1$$

**Content loss:** keep the translation pixel-wise similar to the target

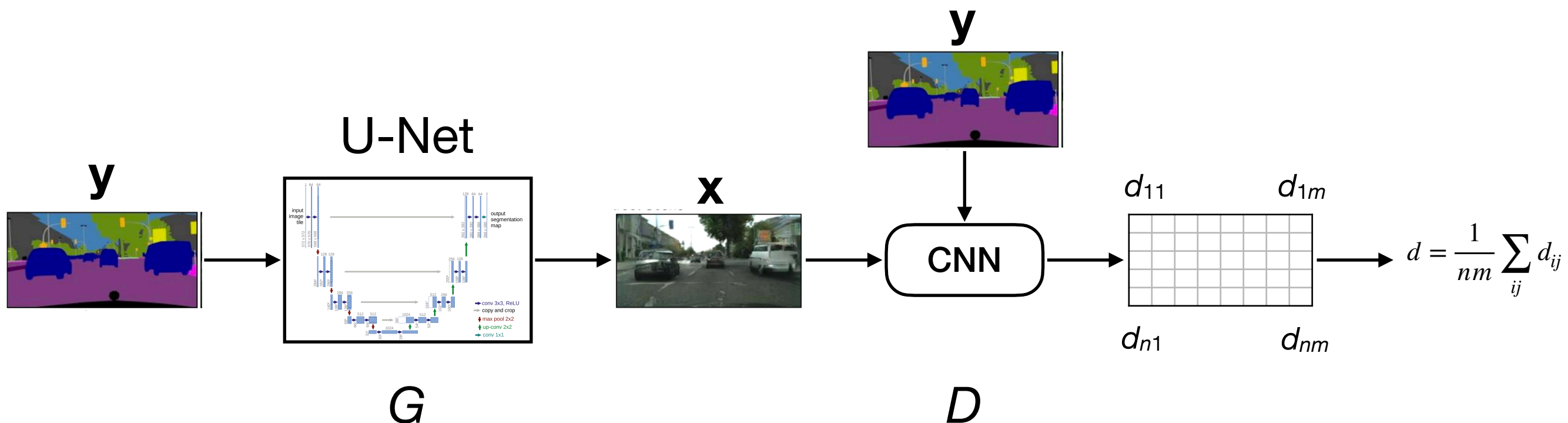


# pix2pix: loss

- $G$  is trained both to maximally confuse  $D$  but also to minimize the  $L_1$  distance between  $\mathbf{x}$  and ground-truth image  $\mathbf{t}$ :

$$f(\mathbf{y}, \mathbf{t}; \theta_G) = f_{\text{cGAN}}(G(\mathbf{y}), \mathbf{y}, D) + \|G(\mathbf{y}) - \mathbf{t}\|_1$$

**cGAN/adversarial loss:** ensure the generated image can fool the discriminator



# Both losses are needed

- $L_1$  loss alone produces blurry images but they are semantically consistent with the input.
- cGAN loss alone produces sharp images but they may contain extraneous objects.
- $L_1 + \text{cGAN}$  gives best results for image generation.

